



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Татјана Гавриловић

**Пројектовање и имплементација
текстуалног претраживача над
транскриптима Youtube подкаста на
српском језику**

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Татјана Гавриловић	Број индекса:	R2-38/2023
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Бранко Милосављевић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Пројектовање и имплементација текстуалног претраживача над транскриптима Youtube подкаста на српском језику
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - мастер рад
Аутор, АУ:	Татјана Гавриловић
Ментор, МН:	Др Бранко Милосављевић, редовни професор
Наслов рада, НР:	Пројектовање и имплементација текстуалног претраживача над транскрипцијама Youtube подкаста на српском језику
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	11/79/48/1/37/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Бранко Милосављевић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Tatjana Gavrilović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Design and implementation of a text search engine over transcripts of YouTube podcasts in the Serbian language
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	11/79/48/1/37/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Изазови обраде српског језика као природног језика	3
2.1	Историјски разлози за ограничене језичке ресурсе	3
2.2	Писмо, дијалекти и фонолошке варијанте	4
2.3	Морфолошка сложеност српског језика	4
2.4	Вишезначност и семантичка сложеност	5
3	Преглед приступа	7
3.1	Припрема података и индексирање	7
3.2	Обрада корисничког упита	8
4	Архитектура система	9
4.1	База података — <i>PostgreSQL</i> и <i>pgvector</i>	9
4.2	Претраживачки систем — <i>Elasticsearch</i>	9
4.3	Позадински сервис — <i>FastAPI</i>	10
4.4	Кориснички интерфејс — <i>Angular</i>	10
4.5	Мрежа и покретање система	11
5	Транскрипција и припрема корпуса	13
5.1	Обим корпуса	13
5.2	Прикупљање и транскрипција	13
5.3	Формат складиштења	14
5.4	Складиштење у <i>PostgreSQL</i> бази	15
5.5	Модел података	15
5.6	Функција за увоз података у базу	16
6	Лексичка претрага (<i>Elasticsearch</i>)	19
6.1	Шта је <i>Elasticsearch</i>	19
6.2	<i>Elasticsearch</i> наспрам релационе базе података	19
6.3	Дистрибуирана архитектура	19
6.4	Како <i>Elasticsearch</i> чува и претражује податке?	20
6.5	BM25 и рангирање резултата	20
6.6	Ниво вршења индексирања	21
6.7	Мапирање	21
6.8	Изазови српског језика у <i>Elasticsearch</i> -у	23
6.9	Анализатор	23
6.10	Алгоритам индексирања	26
6.11	Алгоритам претраге	29
7	Семантичка (векторска) претрага	33
7.1	Векторске репрезентације текста	33
7.2	Мјера сличности: <i>dot product</i> , <i>cosine similarity</i> и еуклидско растојање	34
7.3	Нормализација: привођење свих мјера на исти поредак	35
7.4	Проблем приближне претраге и HNSW	35
7.5	<i>pgvector</i> као векторска база података	36
7.6	Избор <i>embedding</i> модела	36
7.7	Текстуални префикси	36
7.8	Грануларност и <i>chunking</i>	37
7.9	Припрема базе — HNSW индекс	37

7.10	Алгоритам индексирања (<i>offline pipeline</i>)	38
7.11	Алгоритам претраге (online pipeline)	41
7.12	Ограничења векторске претраге	44
7.13	Могућа унапређења	45
8	Хибридизација резултата претраге (RRF)	47
8.1	<i>Reciprocal Rank Fusion</i>	47
8.2	Избор RRF-а у односу на алтернативне приступе	48
8.3	Архитектура — три ендпоинта	49
8.4	Ниво фузије резултата	49
8.5	Алгоритам фузије	49
8.6	Пагинација хибридних резултата	52
8.7	Кеширање	52
8.8	Филтрирање након фузије	53
8.9	Претрага без текстуалног упита	54
8.10	Приказ релевантних исјечака по епизоди	54
8.11	Ограничења и правци даљег развоја	55
9	Евалуација система	57
10	Изглед апликације	59
11	Закључак	61
	Списак слика	63
	Списак листинга	65
	Списак табела	67
	Списак коришћених скраћеница	69
	Списак коришћених појмова	71
	Биографија	75
	Литература	77

Енорман раст и популаризација подкаста (енг. *podcasts*) и *YouTube* садржаја су резултат комбинације технолошке демократизације, промјене навика корисника, те развоја нових начина дистрибуције и зараде. Међутим, док количина доступног садржаја континуирано расте, могућност прецизног проналажења релевантних тренутака унутар дугих епизода заостаје за овим развојем. Корисник који тражи конкретну информацију у епизоди тренутно нема начин да избјегне преслушавање читавог садржаја, нити да добије хронолошки организован преглед тема унутар епизоде са могућношћу директног преласка на релевантан исјечак.

Слична рјешења овог проблема рађена су над корпусом енглеског језика. Платформа *Spotify* је развила семантичку претрагу подкаста засновану на моделу обраде природног језика (ОПЈ, енг. *Natural Language Processing, NLP*) и приближној претрази (к) најближих сусједа (енг. *Approximate Nearest Neighbor, ANN*) [1], ипак ријеч је искључиво о семантичкој претрази, не о хибридном приступу који комбинује семантичку и лексичку претрагу. Пројекат *TREC Podcasts Track* је представљао истраживачки оквир за претрагу подкаст сегмената, покренут 2020. године и настављен 2021. године [2], али је обустављен, а одговарајући скуп података је укинут 2023. године [3].

Иако постоје примјери примјене семантичке претраге на српском језику у другим доменима (нпр. правна пракса [4]) и активан истраживачки рад на текстуалним *NLP* ресурсима за српски (нпр. *COMtext.SR* [5]), ни у једном пронађеном извору није идентификовано рјешење за семантичку или хибридну претрагу говорног, односно подкаст садржаја на српском језику са мапирањем на конкретан тренутак у оквиру епизоде.

Посебан изазов представља и сам српски језик, који припада језицима са ограниченим ресурсима у области обраде природног језика. Доступност квалитетних и обимних корпуса за српски знатно је мања у односу на доминантне језике. Због тога вишејезични модели, кроз пренос знања из других језика, не могу у потпуности обухватити језичке специфичности српског језика, укључујући његову морфосинтаксичку сложеност и различите језичке варијанте [6].

Циљ овог рада је дизајн инфраструктуре и имплементација система за хибридну претрагу *YouTube* садржаја на српском језику, који комбинује лексичку и семантичку претрагу путем методе реципрочне фузије рангова (енг. *Reciprocal Rank Fusion, RRF*), уз очување грануларности на нивоу једног или више сегмената ради навигације до тачног или приближног тренутка у епизоди.

Сљедеће поглавље разматра специфичне изазове обраде српског језика као природног језика, са освртом на историјске околности развоја језичких ресурса, те његово морфолошко богатство које утиче на изградњу система за претрагу. Треће поглавље даје преглед цјелокупног приступа и представља основне токове система — од припреме и индексирања података до обраде корисничког упита и добијања коначних резултата — чиме се пружа оквир за детаљнији опис у наредним поглављима. У глави четири описана је архитектура система, коришћене технологије и организација контејнеризованог окружења. Глава пет посвећена је транскрипцији и припреми корпуса, укључујући поступак добијања транскрипата, тако и дизајн структуре података чуване у релационој бази. У шестом поглављу описује се лексичка претрага заснована на *Elasticsearch* систему, док се поглавље седам бави семантичком, односно векторском претрагом. Глава осам представља поступак хибридизације резултата лексичке и семантичке претраге коришћењем методе *Reciprocal Rank Fusion (RRF)*, тиме би главу девет посветили евалуацији система. Поглавље десет приказује кориснички интерфејс и начин интеракције корисника са системом. Закључним, једанаестим поглављем, заокружиће се предмет овог рада, сумираће се постигнути резултати и указати на могуће правце даљег развоја.

Глава 2

Изазови обраде српског језика као природног језика

Обрада природних језика (енг. *Natural Language Processing, NLP*) се бави рачунарском обрадом и разумјевањем текста написаног на природним језицима, тј. језицима које људи користе у међусобној комуникацији (нпр. српски језик, енглески, итд.) За разлику од програмских језика, код којих су начини изражавања унапријед строго дефинисани, природни језици немају таквих инхерентних ограничења [7].

Развој *NLP* технологија дуго је био усмјерен првенствено на енглески језик, који и данас има доминантну улогу у *NLP* истраживањима [8]. Ова ситуација је дјелимично посљедица доминантне улоге енглеског језика у међународној комуникацији, што је довело и до веће комерцијалне заинтересованости за развој *NLP* рјешења за енглеско говорно подручје [7]. Посљедице су посебно изражене код језика попут српског, за које не постоје ресурси по обиму и квалитету упоредиви са онима доступним за енглески језик. Недостатак великих и језички анотираних корпуса отежава развој и евалуацију *NLP* модела, како класичних статистичких метода, тако и савремених дубоких неуронских мрежа [6].

Корпус представља велику, структурисану збирку аутентичних језичких података — писаних или говорних — представљених у машински читљивом формату, намјенски одабраних и узоркованих тако да буду репрезентативни за одређени језик, и често анотираних језичким метаподацима [6]. Управо величина и квалитет доступног корпуса директно одређују колико поуздано *NLP* систем може да ради над датим језиком — питање које је, како ће бити показано у наставку овог поглавља, за српски језик посебно изражено.

2.1 Историјски разлози за ограничене језичке ресурсе

Мали обим доступних дигиталних ресурса за српски језик није случајност. Оно је посљедица вишевијековног низа историјских околности које су директно утицале на очување писаног наслеђа.

Српски је индоевропски језик који је развио богату средњовјековну књижевну традицију, укључујући темељна дјела попут Мирослављевог јеванђеља (око 1180.), Вукановог јеванђеља (око 1202.), Светосавског номоканона (1219.) и Душановог законика (1349.). Међутим, османска окупација (средина 15. – почетак 19. вијека) разорила је ову традицију. Манастирске библиотеке, централне за производњу рукописа, суочиле су се са уништавањем и пљачком, при чему су рукописи спаљивани, крадени или продавани колекционарима у Русији, Аустрији или Венецији¹ [6].

Након ослобођења од османске окупације (1804–1815), Србија је постепено обнављала своју књижевну и културну инфраструктуру, али је њено текстуално наслеђе наставило да трпи ударце. Балкански ратови (1912–1913) и Први свјетски рат изазвали су значајна разарања, док је најкатастрофалнији губитак услиједио 1941. године, када су њемачке бомбе уништиле Народну библиотеку Србије, уништивши преко 500.000 томова, укључујући 1.424 средњовјековна ћирилична рукописа [6]. Ови губици су дугорочно осиромашили српске језичке и књижевне ресурсе, што данас отежава развој језичких технологија. Недостатак података додатно се продубљује ограниченом дигитализацијом, рестриктивним ауторским правима и недовољном институционалном подршком [6].

¹На примјер, Вуканово јеванђеље данас се налази у Руској националној библиотеци, док је Минхенски српски псалтир однесен у Баварску 1688. године.

2.2 Писмо, дијалекти и фонолошке варијанте

Још један историјски развој са савременим значајем јесте језичка модернизација српског језика током постосманског културног препорода. Српски филолог и етнолог Вук Караџић предводио је свеобухватну језичку реформу почетком 19. вијека, трансформишући не само српски, већ и хрватску језичку стандардизацију. Ово је утицало на будућност српскохрватског језика и ширег јужнословенског језичког простора, при чему је Бечки књижевни договор из 1850. године — који су потписали српски, хрватски и словеначки интелектуалци — прогласио заједнички језик са два писма, ћирилицом и латиницом [6].

Насљеђе овог историјског развоја директно се одражава на језичке особине са којима се овај рад суочава: српски језик карактерише паралелна употреба два писма — ћирилице и латинице — која се у савременој пракси, укључујући и садржај анализиран у овом раду, слободно смјењују, често и унутар истог документа [6]. Поред писма, српски посједује и двије фонолошке варијанте изговора старог словенског гласа јат (ђ) — екавску (нпр. „дете“, „лепо“) и ијекавску (нпр. „дијете“, „лијепо“). Додатно, српски језик обухвата пет дијалеката (призренско-тимочки, косовско-ресавски, шумадијско-војвођански, зетско-јужносанџачки и источнохерцеговачки), те носи препознатљиве морфосинтаксичке особине, попут богате падежне флексије и слободног реда ријечи у реченици [6].

Разлике између дијалеката могу довести до различитих облика истих ријечи у тексту. То је посебно релевантно за призренско-тимочки дијалекат јужне Србије, у којем се, услед историјских језичких контаката на простору Балканског језичког савеза, испољавају морфосинтаксичке варијације повезане са утицајем сусједних језика, укључујући бугарски и македонски. У односу на стандардни српски, нестандартне варијанте призренско-тимочног дијалекта могу показивати различите површинске облике, што додатно отежава њихову аутоматску обраду и претрагу [9]. Уколико би се међу говорницима у корпусу нашли и говорници оваквих регионалних варијетета, прилагођени анализатор развијен за стандардни српски (поглавље 6) не би нужно исправно нормализовао такве облике. Ова могућност није посебно испитивана у оквиру овог рада — наводи се као теоретски, а не емпиријски потврђен изазов. На примјер, у призренско-тимочким говорима јавља се облик „работим“, који се разликује од стандардног српског облика „радим“, иако има исто значење. Овакве разлике могу отежати претрагу текста уколико систем није прилагођен дијалекатским варијантама.

Варијабилност писма имала је директан утицај на имплементацију система описаног у овом раду. И претрага у *Elasticsearch*-у (поглавље 6) и обрада текста прије векторског уграђивања (поглавље 7) захтијевале су нормализацију писма, односно превођење садржаја у јединствени облик (латиницу) прије индексирања. На тај начин се исти појам, без обзира на то да ли је написан ћирилицом или латиницом, третира као један термин, а не као два различита облика.

Латинично писмо српског језика користи дијакритичке знакове, односно додатне ознаке којима се основно слово разликује од другог слова. У српском језику такве знакове имају слова ч, ћ, ш, ж и ђ у латиничном облику: *č, ć, š, ž* и *đ*. Ћирилица за исте гласове има посебна слова, па се они у њој не записују помоћу дијакритичких знакова. Ова разлика је значајна за претрагу јер исти појам може бити записан са дијакритичким знаковима или без њих.

Због тога је нормализација дијакритика укључена у оба дијела система. У оквиру лексичке претраге (поглавље 6), њихова обрада реализована је кроз прилагођени анализатор заснован на ICU додатку [10]. У припреми текста за векторско уграђивање (поглавље 7), нормализација дијакритика представљала је један од првих корака обраде транскрипта, одвојен од конверзије ћирилице у латиницу. На тај начин се смањује број различитих облика истог појма и омогућава његово препознавање без обзира на начин записивања.

2.3 Морфолошка сложеност српског језика

Морфологија представља дио граматике који се бави врстама ријечи, њиховом структуром и промјеном њихових облика [11]. У српском језику посебно су значајне промјенљиве врсте ријечи, код којих се облик

мијења у зависности од различитих граматичких категорија. Међу њима се издвајају именице, придјеви и глаголи, чија морфолошка својства доприносе великој варијабилности површинског облика текста.

Именице имају граматичке категорије рода, броја и падежа [12]. Њихова промјена по падежима назива се деклинација, при чему једна лексема може имати више различитих облика. Лексема је основна самостална јединица лексичког система која обухвата све граматичке облике и различита значења која једна ријеч може остварити [13]. Српски језик има седам падежа: номинатив, генитив, датив, акузатив, вокатив, инструментал и локатив. На примјер, именица град може се појавити у облицима град, града, граду, град, граде, градом, граду. Иако се површински облици разликују, они представљају различите облике исте лексеме. Именице припадају једном од три граматичка рода — мушком, женском или средњем [12].

Придјеви се са именицом уз коју стоје слажу у роду, броју и падежу. Због тога и они мијењају свој облик у зависности од граматичког контекста. На примјер, придјев млад јавља се у облицима млад човјек, млада жена, младо дијете, а кроз падеже у конструкцији млад човјек, младог човјека, младом човјеку, младим човјеком. Осим тога, код описних придјева постоји и могућност степеновања, нпр. висок, виши, највиши, што представља додатни извор морфолошке варијабилности [12].

Глаголи имају сложенији систем морфолошких категорија. Њихови облици зависе од лица и броја, времена, начина и глаголског вида, а српски језик има и различите неличне глаголске облике. Тако се глагол радити може појавити као радим, радиш, ради, радимо, радите, раде, док се кроз различита времена и облике јављају, на примјер, радио је, радиће, радио бих, радећи. Посебну категорију представљају глаголски придјеви, укључујући радни (радио, радила, радило) и трпни (урађен, урађена, урађено). Глаголе такође карактерише глаголски вид, при чему се разликују свршени и несвршени глаголи, нпр. писати – написати [12].

Оваква морфолошка разноврсност значи да се иста лексема у тексту може појавити у великом броју различитих површинских облика. То представља важан изазов за аутоматску обраду и претрагу текста на српском језику, јер облик који се појављује у корисничком упиту не мора бити идентичан облику исте лексеме који се налази у тексту.

Дио ове варијабилности ријешен је у оквиру лексичке претраге описане у поглављу 6, гдје прилагођени анализатор, уз стематизацију (енг. *serbian_stemming*), своди различите облике исте лексеме на заједнички стем. Стематизација представља поступак свођења различитих облика ријечи на њихову заједничку основу, односно стем, који се добија уклањањем одређених наставка и дијелова ријечи и не мора представљати стварну ријеч у језику. На тај начин се, на примјер, различити падежни облици исте именице могу повезати у претрази. Ипак, стематизација не може повезати ријечи које имају сродно значење, али различиту основу, као што су писати и написати. Оне представљају различите лексеме, па их стематизација не може повезати само на основу њихове семантичке сличности. Управо тај преостали дио варијабилности представља један од разлога за увођење семантичке, односно векторске претраге, описане у поглављу 7.

2.4 Вишезначност и семантичка сложеност

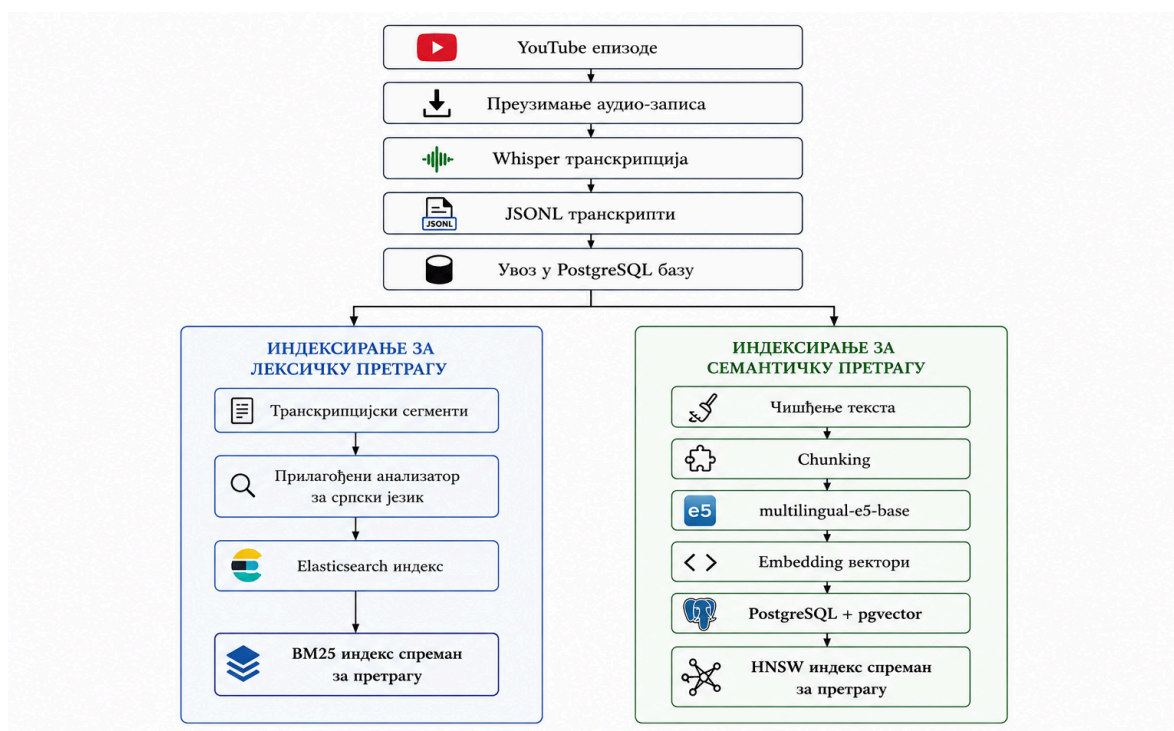
Поред морфолошке флексије, додатан проблем представља чињеница да се текстови на природним језицима често одликују великим степеном комплексности, као и нејасноћама и двосмисленостима, односно вишезначностима у изражавању [7]. Исти концепт може бити изражен различитим ријечима или формулацијама, што представља посебан изазов за лексичку претрагу. На примјер, корисник може претраживати појам „додаци исхрани“, док се у транскрипту подкаста говори о „суплементима“. Иако се ове формулације разликују на нивоу ријечи, њихово значење је блиско. Лексичка претрага заснована на подударану термина такву везу не може поуздано препознати, осим ако су одговарајуће ријечи унапријед наведене као синоними. Ово ограничење представља један од главних разлога за увођење семантичке, односно векторске претраге (поглавље 7), која омогућава поређење текстова на основу њиховог значења, а не само на основу подударану конкретних ријечи.

У овом поглављу је дат преглед цјелокупног приступа проблему, односно приказује начин на који су појединачне компоненте повезане у цјелину, док разматрање имплементационих детаља сљедује у даљим главама рада. Систем се може посматрати кроз два основна тока: припрему података праћену индексирањем и обраду корисничког упита.

3.1 Припрема података и индексирање

Слика 1 приказује ток података од изворног *YouTube* садржаја до индекса припремљених за лексичку и семантичку претрагу. Аудио-садржај преузет са *YouTube* канала пролази кроз транскрипцију (поглавље 5.2), након чега се добијени транскрипти чувају у *JSONL* формату (поглавље 5.3) и увозе у релациону базу података (поглавље 5.4). База затим представља централни извор структурираних података о каналима, епизодама и сегментима транскрипта (поглавље 5.5).

Након увоза података извршавају се два одвојена поступка индексирања: индексирање за лексичку претрагу (Глава 6) и индексирање за семантичку претрагу (Глава 7). У оквиру лексичке претраге, транскрипцијски сегменти пролазе кроз прилагођени анализатор за српски језик (поглавље 6.9), након чега се групно индексирају у *Elasticsearch*-у (поглавље 6.10). На тај начин формира се индекс над којим се релевантност резултата одређује примјеном *BM25* алгоритма (поглавље 6.5). Код семантичке претраге, транскрипцијски сегменти пролазе кроз чишћење (поглавље 7.10.1) и груписање у *chunk*-ове (поглавље 7.8.2), након чега се изабраним *embedding* моделом (поглавље 7.6) претварају у *embedding* векторе. Вектори се чувају у релационој бази помоћу *pgvector* екстензије (поглавље 4.1) и над њима се формира *HNSW* индекс за приближну векторску претрагу (поглавље 7.9). Резултат фазе индексирања су два различита индекса над којима се, након корисничког упита, извршавају лексичка и семантичка претрага приказане у наредном одјелјку.



Слика 1: Преглед припреме података и фаза индексирања

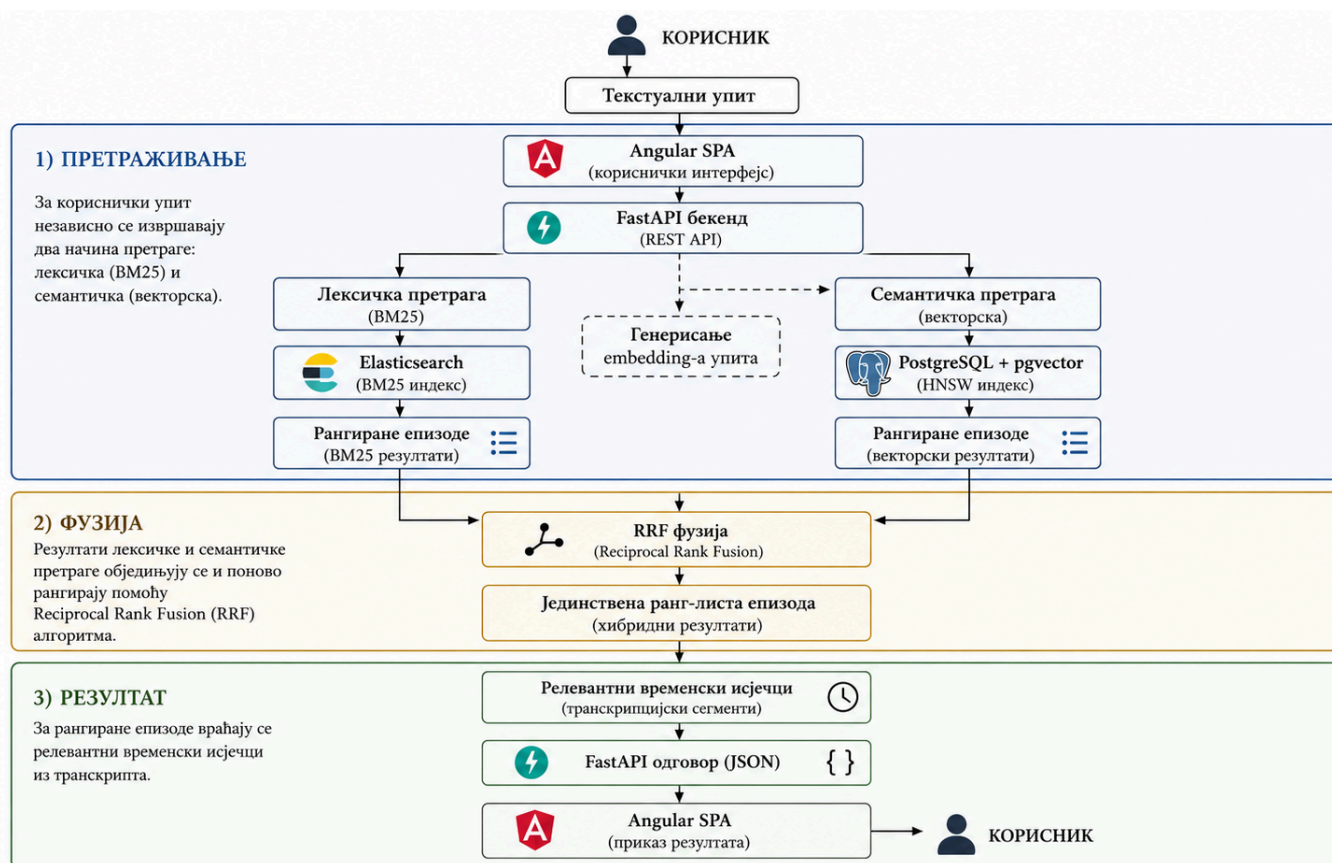
3.2 Обрада корисничког упита

Слика 2 приказује ток система, који се покреће при свакој корисничкој претрази. Текстуални упит унесен кроз *Angular* кориснички интерфејс (поглавље 4.4) просљеђује се *FastAPI* позадинском сервису (поглавље 4.3), након чега се обрађује кроз два независна механизма, кроз лексичку и семантичку претрагу.

Упит се просљеђује *Elasticsearch*-у, који претражује претходно формирани индекс и враћа ранг-листу епизода на основу лексичке релевантности (поглавље 6.11). За извршавање семантичке претраге, неопходно је прво претворити кориснички упит у *embedding* вектор користећи исти *embedding* модел као приликом индексирања, уз употребу префикса „query: “, након чега се над *HNSW* индексом проналазе најсличнији *chunk*-ови и формира ранг-листа епизода (7.11).

Добијене ранг-листе епизода обједињују се методом *Reciprocal Rank Fusion (RRF)*, описаној у глави 8. На основу позиције епизоде у обје листе израчунава се њен коначни *RRF* скор и формира се јединствена хибридна ранг-листа (поглавље 8.5).

За рангиране епизоде прикупљају се и релевантни временски исјечци транскрипта пронађени лексичком и семантичком претрагом. Исјечци се обједињују и хронолошки сортирају, након чега *FastAPI* враћа резултате корисничком интерфејсу у јединственом формату.



Слика 2: Преглед обраде корисничког упита и формирања хибридних резултата

Систем описан у овом раду реализован је као скуп независних сервиса, међусобно повезаних преко заједничке мреже и покренутих помоћу *Docker Compose*-а, који представља алат за дефинисање и покретање апликација састављених од више контејнера једном командом [14]. Контејнеризација омогућава да сваки сервис (база података, претраживачки систем, позадински сервис, кориснички интерфејс) буде изолован у сопственом окружењу, са тачно дефинисаним зависностима, при чему се цијели систем покреће и зауставља на исти начин, без обзира на то да ли се извршава на развојној машини или серверу. Систем чине пет сервиса на заједничкој мрежи (енг. *app-network*): база података (*PostgreSQL* са *pgvector* екстензијом), административни алат за базу (*pgAdmin4*), претраживачки систем (*Elasticsearch*), позадински сервис (*FastAPI*) и кориснички интерфејс (*Angular*).

4.1 База података — *PostgreSQL* и *pgvector*

PostgreSQL у оквиру система представља централну базу података и извор истине (енг. *source of truth*), главни и поуздани извор за све структуриране податке. У њему се чувају метаподаци о каналима, епизодама и епизодним сегментима. Поред тога, захваљујући екстензији *pgvector*, у бази се чувају и векторске репрезентације текста (енг. *embeddings*), које се користе за семантичку претрагу описану у поглављу 7. *pgvector* је екстензија отвореног кода која *PostgreSQL*-у додаје подршку за чување вектора и претрагу по њиховој сличности [15]. Разлози за избор *pgvector*-а у односу на друга рјешења, као што су *FAISS*, *Qdrant*, *Milvus* и *Pinecone*, детаљно су образложени у поглављу 7.

За покретање базе користи се службена слика (енг. *image*) ***pgvector/pgvector:pg16***, која садржи *PostgreSQL* 16 са уграђеном *pgvector* екстензијом. Подаци се чувају у именованом простору за трајно чување података (енг. *volume*), чиме се обезбјеђује њихово очување и након поновног покретања контејнера. Спремност сервиса провјерава се помоћу *healthcheck*-а, који користи команду *pg_isready*. Конфигурација *PostgreSQL* базе са *pgvector* екстензијом се може погледати у листингу 1.

```
postgres:
  image: pgvector/pgvector:pg16
  environment:
    - POSTGRES_DB=podcast_search
    - POSTGRES_USER=podcast_user
    - POSTGRES_PASSWORD=podcast_password
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U podcast_user"]
```

Листинг 1: Конфигурација *PostgreSQL* базе са *pgvector* екстензијом

За административни приступ бази података, који укључује преглед табела и ручно извршавање упита, коришћен је *pgAdmin4*. Административном интерфејсу приступа се путем веб интерфејса на порту 5050.

4.2 Претраживачки систем — *Elasticsearch*

Elasticsearch сервис је заснован на прилагођеној *Docker* слици, изграђеној на службеној слици ***elasticsearch:8.13.2***. Слици је додат *analysis-icu* додатак [10], који је коришћен за нормализацију и обраду ћириличног и латиничног текста, што је детаљније описано у поглављу 6. Прилагођена слика се дефинише следећим *Dockerfile*-ом, приказаним у листингу 2.

```
FROM docker.elastic.co/elasticsearch/elasticsearch:8.13.2
RUN bin/elasticsearch-plugin install --batch analysis-icu
```

Листинг 2: *Dockerfile* за изградњу прилагођене *Elasticsearch* слике са *analysis-icu* додатком

Сервис ради у режиму *single-node*, што значи да се *Elasticsearch* извршава као један чвор, без повезивања са другим чворовима у кластеру. *X-Pack* безбједносни механизми [16], који обезбјеђују аутентификацију и контролу приступа *Elasticsearch*-у, су искључени, што одговара развојном и интерном окружењу у којем је систем коришћен. Спремност сервиса провјерава се помоћу *healthcheck*-а, који чека да статус кластера више не буде ред. Комплетна конфигурација индекса, анализатора и поступака индексирања и претраге описана је у поглављу 6.

4.3 Позадински сервис — *FastAPI*

Позадински дио система реализован је кроз *Python 3.11*, уз коришћење оквира *FastAPI* и *Uvicorn* сервера, који имплементира *ASGI* (енг. *Asynchronous Server Gateway Interface*) стандард и служи за покретање апликације и обраду *HTTP* захтјева. *FastAPI* је изабран због једноставне имплементације *API*-ја. Поред тога, *Python* пружа развијен екосистем за машинско учење и обраду природног језика [17], који је у овом систему коришћен и за генерисање векторских репрезентација помоћу библиотеке *sentence-transformers*.

За рад са базом података користи се *SQLAlchemy* као *ORM* (енг. *Object-Relational Mapping*), уз *psycopg2-binary* као *PostgreSQL* управљачки програм. Промјене структуре базе управљају се помоћу *Alembic*-а, док се за рад са векторским колонама користи *Python* подршка за *pgvector*. Валидација конфигурације и података реализована је помоћу библиотека *Pydantic* и *pydantic-settings*, док се вриједности из окружења учитавају помоћу библиотеке *python-dotenv*. Комуникација са *Elasticsearch*-ом остварује се преко службеног *Python* клијента.

Docker слика позадинског сервиса гради се у једној фази, уз коришћење кеширања слојева. Зависности наведене у датотеци *requirements.txt* инсталирају се прије копирања изворног кода, што омогућава да се приликом измјене кода поново искористи постојећи слој са инсталираним зависностима, без њихове поновне инсталације. Модели преузети преко библиотека *Hugging Face*, укључујући модел за генерисање векторских репрезентација, чувају се у именованом *volume*-у (*hf_cache*). На тај начин се избјегава њихово поновно преузимање приликом поновног покретања контејнера.

4.4 Кориснички интерфејс — *Angular*

Кориснички интерфејс реализован је у *Angular*-у [18], а за његову изградњу користи се *Docker multi-stage build* [19], који раздваја фазу изградње апликације од фазе њеног покретања. У првој фази, заснованој на слици *node:22-alpine*, гради се продукцијска верзија апликације, док се у другој фази добијени статички фајлови копирају у минималну *nginx:alpine* слику. На тај начин коначна слика садржи само фајлове неопходне за покретање апликације, без изворног кода и алата који су потребни само током изградње, чиме се смањује њена величина.

Angular апликација реализована је као *SPA* (енг. *Single Page Application*), што значи да се рутирање између страница обавља на страни клијента. Ово је посебно важно за директан приступ конкретној епизоди и одговарајућем тренутку у транскрипту. Због тога је *nginx* подешен тако да захтјеве за путање које не представљају постојеће статичке фајлове прослиједи апликацији преко *index.html*, приказано у листингу 3.

```
location / {
    try_files $uri $uri/ /index.html;
}
```

Листинг 3: Конфигурација *nginx*-а за просљеђивање захтјева *Angular SPA* апликацији

Овакво подешавање омогућава да директан приступ одређеној рути или освјежавање странице функционише исправно, јер *nginx* захтјев просљеђује *Angular*-овом механизму за рутирање умјесто да врати грешку 404. Додатно, *nginx* је подешен за компресију одговора помоћу *gzip*-а и кеширање статичких ресурса, као

што су *JavaScript* и *CSS* фајлови, чиме се смањује количина података која се преноси приликом поновног учитавања апликације.

4.5 Мрежа и покретање система

Сви сервиси комуницирају преко заједничке *Docker bridge* мреже (*app-network*), што им омогућава међусобну комуникацију преко имена сервиса (нпр. позадински сервис приступа бази преко хоста *postgres*, а не *localhost-a*), без потребе за излагањем интерних портова ван контејнера. Покретање и заустављање, као и потпуно брисање базе приказане су у листингу 4 кроз њихове команде.

```
# Покретање свих сервиса у позадини уз поновну изградњу слика
docker-compose up -d --build

# Заустављање и уклањање контејнера уз задржавање података у volume-има
docker-compose down

# Заустављање и уклањање контејнера и именованих volume-a
docker-compose down -v
```

Листинг 4: Команде за покретање и заустављање *Docker Compose* сервиса

Глава 5

Транскрипција и припрема корпуса

Ово поглавље описује поступак прикупљања и припреме корпуса који представља основу за претрагу описану у наредним поглављима. Најприје је представљен обим и извор прикупљених података, а затим је описан поступак транскрипције видео садржаја. Након тога, објашњен је начин организације и складиштења добијених података на диску, као и њихов увоз у релациону базу података која се користи у систему.

5.1 Обим корпуса

Корпус коришћен у овом раду прикупљен је са 32 *YouTube* канала на српском језику, који обухватају различите тематске области и формате, укључујући интервјуе, друштвено-политичке подкасте, едукативни и научно-популарни садржај, као и забавне формате. Укупно је прикупљено и транскрибовано 15.889 епизода.

Избор канала обухвата разноврсне теме, говорнике и стилове комуникације, чиме се обезбјеђује хетерогеност корпуса и омогућава испитивање система у условима различитим од оних који би били присутни у једној уско дефинисаној тематској области. На тај начин корпус пружа погодну основу за процјену способности система да претражује садржај различитих врста и тематских усмјерења. Табела 1 приказује преглед свих канала обухваћених корпусом, заједно са бројем епизода прикупљених са сваког канала.

Табела 1: Списак канала у корпусу и број транскрибованих епизода по каналу.

Канал	Број епизода	Канал	Број епизода
Без Цензуре	3.345	РТС Образовно-научни програм	2.105
РТС Око	1.725	Анђеласт	1.073
Појачало	701	Приче У Нама	701
РТС ТВ серије	828	РТС Ординација	789
Да сам ја неко	656	Никола Радин Подкаст	501
Иван Косогор Подкаст	353	РТС Квадратура круга	353
Катена мунди	340	Бизнис Приче	324
Нови Стандард	289	Јао Миле	244
Још подкаст један	234	Центар за антиауторитарне студије	157
Рубикон	119	Тампон зона	119
Вредно приче	115	Миљанов корнер	102
Сто минута буке	168	Два и по психијатра	90
Лукавство ума	85	Изокренута учионица	76
Рекетирање	75	Сада и овде	72
Другачији фитнес	49	Контранапад	46
Залет иновација	29	Ћао Невена	26

5.2 Прикупљање и транскрипција

Видео садржај преузет са *YouTube* канала било је потребно претворити у текстуални облик погодан за даљу обраду. У ту сврху коришћена је технологија аутоматског препознавања говора (енг. **Automatic Speech Recognition**, **ASR**), која представља област вјештачке интелигенције усмјерену на претварање говорног

сигнала у текстуални облик. Један од савремених система заснованих на овој технологији је **Whisper**, модел за аутоматско препознавање и транскрипцију говора који је развила компанија *OpenAI*. Модел је обучен на великом вишејезичном скупу аудио података и намјењен је аутоматској транскрипцији говорног садржаја, уз подршку за различите језике и облике говора [20]. Наведене карактеристике биле су посебно значајне за избор **Whisper**-а у овом раду, будући да се обрађује говорни садржај различитих говорника, при чему се могу јавити варијације у изговору, акценту, интонацији и присуство позадинске буке.

5.3 Формат складиштења

Резултат транскрипције за сваку епизоду складиштен је у *JSONL* (енг. *JSON Lines*) формату — текстуалном формату у којем сваки ред датотеке представља један самосталан и ваљан *JSON* објекат, за разлику од класичног *JSON* формата, у којем цијела датотека мора представљати један објекат или низ објеката.

Овакав приступ омогућава инкрементално читање и обраду датотеке ред по ред, без потребе да се цјелокупна датотека учита у меморију одједном. То је посебно значајно у овом случају, будући да један ред, односно једна епизода, укључује цјелокупан транскрипт са временски означеним (енг. *timestamp*-ованим) сегментима, па датотека која садржи хиљаде епизода једног канала може достићи знатну величину.

Грануларност складиштења је на нивоу канала: **један канал = једна .jsonl датотека**, при чему сваки ред одговара једној епизоди тог канала. Ове датотеке се додатно компресују помоћу *gzip* алгорита (*.gz* екстензија), чиме се значајно смањују потребан простор на диску и вријеме преноса, уз занемарљив трошак декомпресије приликом увоза података у базу.

У листингу 5 је дат скраћени приказ структуре једног реда, односно једне транскрибоване епизоде. Поља *description*, *plain_text* и низ *timestamped_segments* са низом *words* скраћени су ради прегледности. Пуна структура садржи цјелокупан транскрипт, као и временске ознаке за сваку изговорену ријеч.

```
{
  "video_id": "upxT3PGinY0",
  "title": "Branislav Ristivojević – Kosovo i Metohija je ogledalo srpske politike / Rubikon podcast 105",
  "description": "Гост емисије Рубикон био је... [скраћено]",
  "published_at": "2025-06-12T17:01:10Z",
  "duration": "PT1H4M28S",
  "view_count": "58228",
  "like_count": "443",
  "channel_title": "Rubikon",
  "channel_id": "UCb0hTyq2vHyxaimv5cgjAdw",
  "tags": [],
  "file_path": "/var/youtube/rubikon/upxT3PGinY0.webm",
  "download_time": "2025-08-28T20:15:34.711178",
  "url": "https://www.youtube.com/watch?v=upxT3PGinY0",
  "transcription": {
    "plain_text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri... [скраћено]",
    "timestamped_segments": [
      {
        "id": 0,
        "start": 0.0,
        "end": 2.58,
        "text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri.",
        "words": [
          {
            "word": " Nama",
            "start": 0.0,
            "end": 0.16,
            "probability": 0.93
          }
        ]
      }
    ]
  }
}
```

Листинг 5: Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде

Као што се види, један ред садржи метаподатке епизоде (наслов, опис, интернет адресу, датум објаве, број прегледа и лајкова), као и цјелокупан резултат транскрипције — његов садржај (*plain_text*) и низ временски означених сегмената (*timestamped_segments*). *Whisper* у свом изворном излазу пружа и детаљнији ниво грануларности — за сваки сегмент доступан је и низ појединачних ријечи, при чему свака ријеч има сопствени временски опсег и вјероватноћу препознавања (*probability*). Међутим, овај ниво детаља се намјерно не преноси у базу података приликом увоза. Задржавају се само временски опсег (*start, end*) и текст на нивоу цјелокупног сегмента.

Оваква грануларност на нивоу сегмената транскрипта омогућава касније прецизно позиционирање корисника на тачан тренутак у видеу приликом претраге. На тај начин резултат претраге не упућује само на одговарајућу епизоду, већ може корисника усмјерити и на конкретан дио видеа у којем се тражени садржај појављује.

5.4 Складиштење у PostgreSQL бази

Подаци из *.jsonl.gz* датотека увозе се и трајно складиште у PostgreSQL релационој бази података. PostgreSQL овдје представља извор истине (енг. *source of truth*) за структуриране податке о каналима, епизодама и сегментима транскрипта. Детаљи о самом сервису, контејнеризацији и екстензији *pgvector* — која се касније користи за векторску претрагу (поглавље 7) — већ су обрађени у поглављу 4.

5.5 Модел података

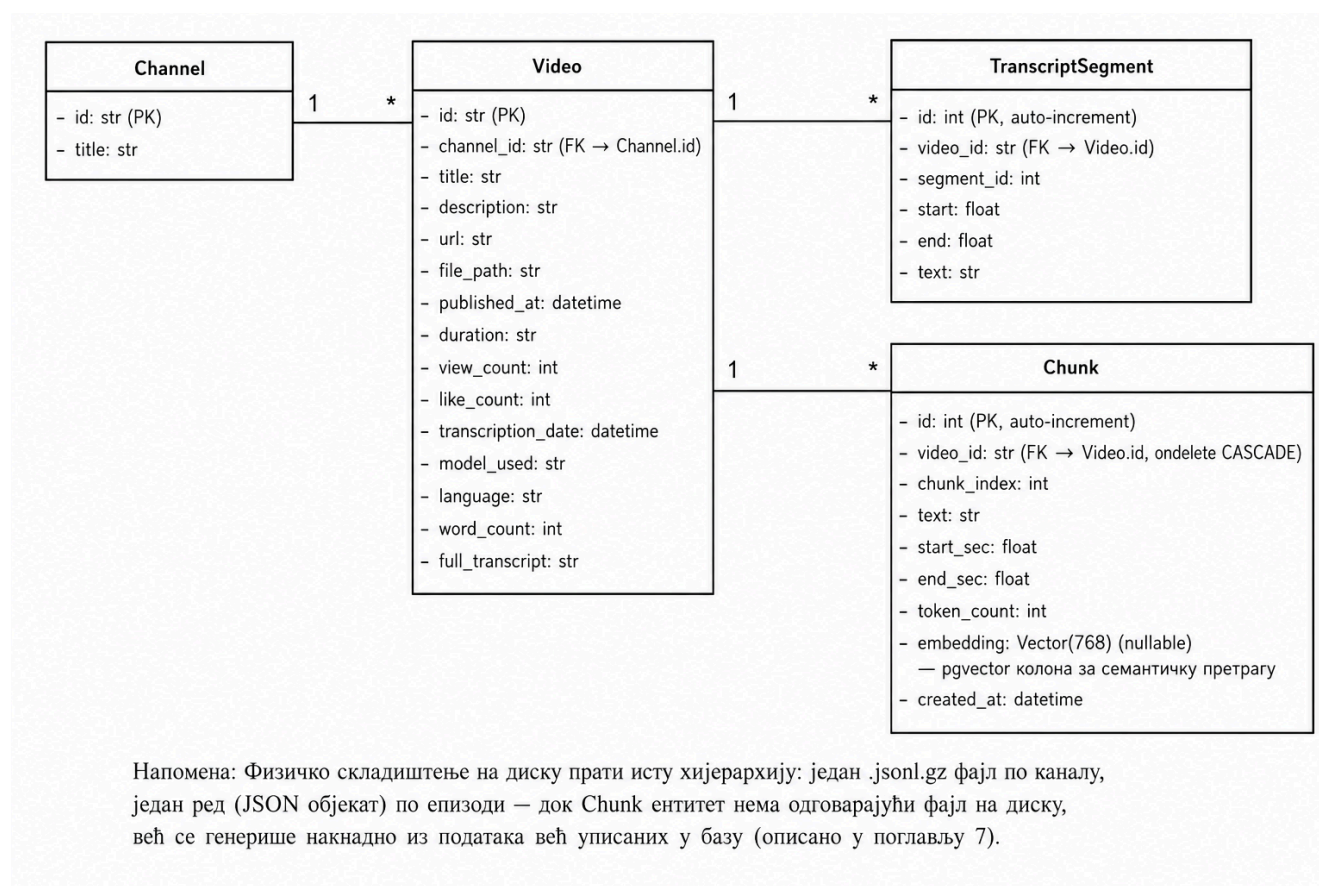
Подаци увезени у базу организовани су кроз четири повезана ентитета, који одражавају природну хијерархију изворних *.jsonl.gz* датотека, уз један додатни ентитет који не потиче директно из *.jsonl* формата, већ се генерише накнадно за потребе семантичке претраге:

- Канал (*Channel*) — један запис за сваки YouTube канал;
- Видео епизоде (*Video*) — један запис за сваку транскрибовану епизоду, повезан са тачно једним каналом;
- Сегмент транскрипта (*TranscriptSegment*) — један запис за сваки временски означени дио транскрипта, повезан са тачно једном епизодом;
- *Chunk* — један запис за сваку текстуалну цјелину насталу груписањем узастопних сегмената транскрипта, намијењену генерисању векторских уграђивања (енг. *embedding*) која се користе у семантичкој, односно векторској, претрази (детаљно објашњено у поглављу 7).

Другим ријечима, структура података на диску — једна *.jsonl.gz* датотека по каналу, један ред по епизоди и више сегмената унутар транскрипта — директно се пресликава у релациони модел. Један канал има више видеа (1:N), док један видео има више сегмената транскрипта (1:N) и, независно од тога, више *chunk*-ова (1:N).

Табела *Chunk* садржи и колону *embedding* типа *Vector(768)* (*pgvector* тип, поглавље 4), као и јединствени (*unique*) индекс над паром *video_id, chunk_index*, који гарантује јединственост редосљеда *chunk*-ова унутар једне епизоде.

Дијаграм класа на слици 3 приказује сва четири ентитета, њихове кључне атрибуте и везе између њих.

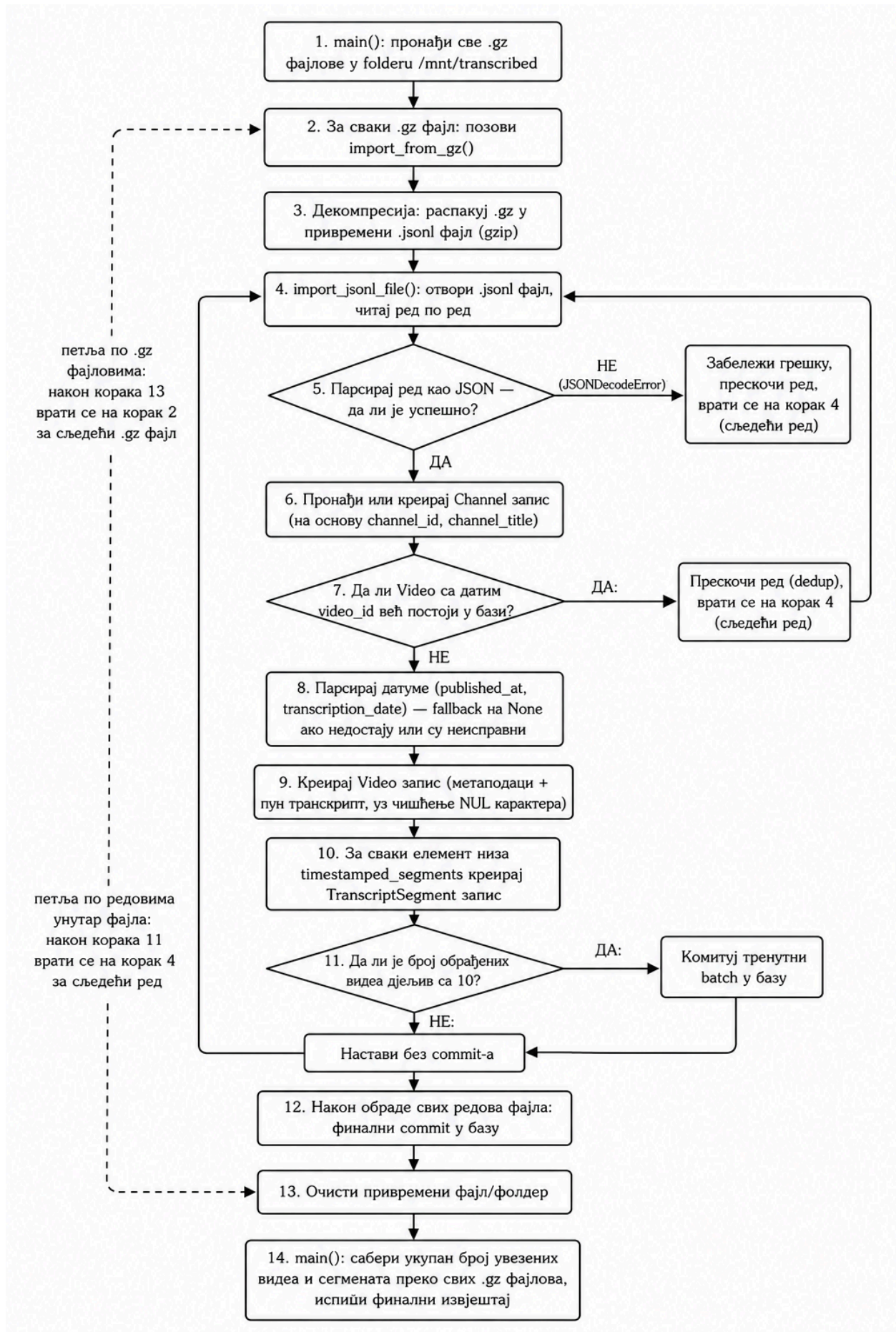
Слика 3: Дијаграм класа модела података: *Channel*, *Video*, *TranscriptSegment* и *Chunk*.

5.6 Функција за увоз података у базу

Увоз података из *.jsonl.gz* датотека у *PostgreSQL* базу обавља се скриптом *import_data.py*, која се извршава једнократно или по потреби, приликом додавања нових или ажурираних датотека. Скрипта проналази све *.gz* датотеке у фолдеру */mnt/transcribed*, декомпресује их у привремене *.jsonl* датотеке и редом обрађује њихов садржај. Током обраде провјерава се исправност *JSON* записа и постојање видеа у бази, чиме се неисправни редови могу прескочити, а поновни увоз истих података не доводи до дуплирања. За нове записе креирају се одговарајући ентитети ***Channel***, ***Video*** и ***TranscriptSegment***, док се промјене периодично потврђују у трансакцији ради ефикаснијег рада. Након обраде свих датотека привремени подаци се уклањају, а скрипта формира завршни извјештај о броју увезених видеа и транскрипционих сегмената. Комплетан ток поступка приказан је на слици 4, а скрипта се покреће унутар *backend* контејнера командом приказаном у листингу 6.

```
docker-compose exec backend python -m app.import_data
```

Листинг 6: Покретање скрипте за увоз података унутар *backend* контејнера

Слика 4: Алгоритам увоза података из *.jsonl.gz* датотека у *PostgreSQL* базу.

Лексичка претрага (*Elasticsearch*)

Лексичка претрага (енг. *lexical search*), позната и као претрага по кључним ријечима (енг. *keyword search*) или пуна-текстуална претрага (енг. *full-text search*), је приступ претраживања текста заснованог на директном подударану термина из корисничког упита са термима садржаним у документима. Лексичка претрага се ослања на статистичке методе у обради природног језика као што је *TF-IDF* (енг. *Term Frequency-Inverse Document Frequency*) и његовог унапређења *Okapi BM25* (енг. *Best Matching 25, BM25*). *TF-IDF* мјери важност и релевантност одређене ријечи унутар неког документа у односу на цијелу колекцију докумената [21], док *BM25* рангира документе на основу три фактора: учесталости термина упита у документу, ријеткости тог термина у цијелом корпусу (обрнута учесталост документа) и дужине документа у односу на просјечну дужину докумената у колекцији [22]. У оквиру овог рада, лексичка претрага имплементирана је помоћу система *Elasticsearch*.

6.1 Шта је *Elasticsearch*

Elasticsearch је систем намјењен за брзо претраживање и анализу великих количина података у реалном времену. Омогућава претраживање различитих врста података, од обичног текста до структурираних података, а може се користити и за њихову анализу. Његова предност је у томе што омогућава да се велике количине информација претражују брзо и ефикасно, због чега се користи у различитим познатим системима. Википедија (енг. *Wikipedia*) га користи за претрагу текстуалног садржаја и давање приједлога корисницима. Новинарској апликацији, *The Guardian*, служи за праћење реакција публике на нове чланке, док страница *Stack Overflow* помоћу њега проналази слична питања и одговоре. Платформа *GitHub* га користи за претраживање огромне количине програмског кода [23].

6.2 *Elasticsearch* наспрам релационе базе података

Релационе базе података (нпр. *PostgreSQL*) првенствено су намијењене поузданом чувању и обради структурираних података, као и извршавању прецизно дефинисаних упита. Иако *PostgreSQL* подржава претрагу пуног текста, таква претрага није његова примарна намјена и захтијева додатну конфигурацију и одговарајуће индексе. Код система чији је основни задатак претрага великих количина текстуалног садржаја, јавља се потреба за напреднијом анализом текста и ефикаснијим рангирањем резултата.

За разлику од релационих база, *Elasticsearch* је специјализован за претрагу и анализу текста. Он омогућава индексирање текстуалних докумената, њихову лексичку анализу и рангирање резултата према релевантности упита. Поред тога, подржава механизме који омогућавају флексибилнију претрагу, попут толерисања типографских грешака и различитих облика ријечи. Због тога се *Elasticsearch* показује као погодније рјешење за дио система који је задужен за претрагу великог корпуса текстуалних докумената, док релациона база и даље задржава своју улогу у чувању и управљању структурираним подацима.

6.3 Дистрибуирана архитектура

Elasticsearch је дистрибуиран систем који се састоји од једног или више чворова (енг. *nodes*) повезаних у кластер (енг. *cluster*). Чвор представља покренуту инстанцу *Elasticsearch-a*, док кластер представља групу чворова који заједнички управљају подацима и ресурсима система. Чворови који припадају истом кластеру идентификују се помоћу заједничког назива кластера (*cluster.name*). *Elasticsearch* може функционисати са само једним чвором, при чему такав чвор истовремено представља цијели кластер [23].

Фрагмент (енг. *Shard*) представља физичку јединицу у којој *Elasticsearch* складишти дио података једног индекса. Индекс је логичка цјелина која може бити састављена од једног или више фрагмената, док се сами

документи физички чувају и индексирају унутар фрагмената. Сваки *shard* представља засебну инстанцу претраживачког механизма и може се посматрати као самостална јединица за складиштење и претрагу [23].

Копија (енг. *Replica*) представља копију примарног фрагмента и служи као додатни ниво заштите података у случају отказа хардвера или недоступности чвора на којем се налази примарни фрагмент. Поред обезбјеђивања редундантности, копија може учествовати и у обради захтјева за читање, као што су претрага и преузимање докумената, чиме се може повећати капацитет система за операције читања [23].

Фрагмент и копија се постављају у конфигурацији индекса. У случају постављања *Elasticsearch*-а у дистрибуирано *Cloud* окружење, уз коришћење *Kubernetes* кластера са више чворова, могуће је распоредити *Elasticsearch* чворове и фрагменте на више физичких или виртуелних машина. На тај начин различити фрагменти могу бити смјештени на различитим чворовима, па се упит који обухвата више фрагмената може обрађивати паралелно. Оваква архитектура омогућава хоризонтално скалирање система и расподјелу оптерећења на више рачунарских ресурса, што може побољшати перформансе претраге код великих количина података и већег броја истовремених захтјева. У оквиру овог мастер рада, *Elasticsearch* се извршава локално на једном чвору, што је довољно за развој и тестирање система.

6.4 Како *Elasticsearch* чува и претражује податке?

Један од основних концепата на којем се заснива ефикасна претрага текста у *Elasticsearch*-у јесте инвертовани индекс (енг. *inverted index*) [23]. За разлику од класичног приступа, у којем се полази од документа и у њему траже одговарајуће ријечи, инвертовани индекс успоставља обрнуту везу, односно за сваки термин чува информацију о документима у којима се тај термин појављује [24]. Ова структура се може упоредити са индексом на крају књиге, гдје се уз одређену ријеч наводе странице на којима се она појављује. На сличан начин, *Elasticsearch* за сваки термин памти скуп докумената који га садрже.

Када корисник пошаље упит, *Elasticsearch* не мора пролазити кроз сваки документ појединачно. Умјесто тога, претрага се врши над већ изграђеним индексом, што значајно смањује количину података коју је потребно обрадити. Поред информације о документима у којима се термин појављује, индекс може садржати и додатне информације, попут учесталости термина и његове позиције у документу. Ови подаци се користе за напреднију претрагу и одређивање релевантности резултата [24].

6.5 BM25 и рангирање резултата

Проналажење докумената који садрже термине из корисничког упита представља само први корак лексичке претраге. Уколико више докумената садржи исте термине, потребно је одредити њихову релевантност и поређати их тако да се најрелевантнији резултати прикажу на почетку. У *Elasticsearch*-у се за ту намјену користи алгоритам **Okapi BM25** (енг. **Best Matching 25, BM25**), који је подразумевани модел за израчунавање оцјене релевантности и рангирање докумената [25].

У имплементираном систему *BM25* представља основу за рангирање резултата лексичке претраге. Основни текстуални упит реализован је помоћу **match** упита над пољем **text**, након чега *Elasticsearch* додјељује сваком пронађеном документу одговарајући скор релевантности. На тај начин резултати се не враћају само на основу постојања подударања са упитом, већ се међусобно рангирају према степену релевантности.

У оквиру упита додатно су коришћени услови **must** и **filter**. Услови наведени у **must** дијелу представљају обавезна подударања која учествују у израчунавању скорa, док се **filter** користи за ограничавање скупа докумената без утицаја на њихову релевантност. Оваква подјела омогућава да се критеријуми претраге одвоје од услова који служе за филтрирање резултата.

BM25 у овом раду није имплементиран као посебан алгоритам, већ се користи као уграђени механизам за рангирање који обезбјеђује *Elasticsearch*. На тај начин лексичка претрага користи могућности инвертованог индекса за ефикасно проналажење подударних докумената, док *BM25* одређује њихов релативни поредак према релевантности.

6.6 Ниво вршења индексирања

Једна од првих одлука приликом имплементације лексичке претраге односила се на избор нивоа индексирања података. Разматране су биле три могућности:

- **Један документ по транскрипцији видеа** - организација индекса у којем један документ представља цијели видео;
- **Један документ по сегменту** - организација индекса у којем би транскрипцијски сегмент представљао засебан документ;
- **Један документ по транскрипцији видеа са угњежденим сегментима** - организација индекса у којем би се сегменти чували унутар документа који представља цијели видео;

Иако би индексирање на нивоу видеа било једноставније и омогућило природно рангирање резултата по видеу, такав приступ не би омогућио директно враћање прецизног временског тренутка на којем се тражени садржај појављује. Са друге стране, чување сегмената као засебних докумената омогућава да сваки резултат претраге директно садржи текст и одговарајућу временску ознаку, чиме се кориснику може приказати конкретан дио разговора и омогућити прелазак на одговарајући тренутак у видеу. Због тога је у овом раду изабрана **грануларност на нивоу транскрипцијског сегмента**, односно сваки сегмент представља један документ у *Elasticsearch* индексу. Ова одлука је значајна и за каснију имплементацију семантичке претраге, јер се векторска претрага такође заснива на мањим текстуалним јединицама изведеним из транскрипцијских сегмената. На тај начин оба приступа претрази могу радити над приближно истим нивоом текстуалне грануларности, што поједностављује њихово касније поређење и комбиновање. Трећа могућност - један документ по транскрипцији видеа са угњежденим сегментима - био је избачен гледајући са становишта перформанси. Овакав приступ је сложенији и спорији над великим документима.

6.7 Мапирање

У *Elasticsearch*-у се структура података дефинише помоћу мапирања (енг. *mapping*), који одређује тип и начин индексирања појединачних поља документа. Мапирање се може упоредити са шемом у релационим базама података, гдје структура табеле дефинише типове и карактеристике њених колона. На основу мапирања *Elasticsearch* одређује како ће се вриједности појединих поља чувати, анализирати и индексирати, што директно утиче на начин извршавања и претраге над тим пољима [23].

Приликом дефинисања мапирања *Elasticsearch* индекса потребно је одредити која поља треба индексирати и који тип података им одговара. Основна одлука је била да се текст транскрипцијских сегмената дефинише као **text** поље, јер представља примарни садржај над којим се врши пуна-текстуална претрага (енг. *full-text search*). Насупрот томе, идентификатори и поља која се користе за филтрирање, сортирање или груписање дефинисани су одговарајућим нетекстуалним типовима. На овај начин мапирање је прилагођено стварним потребама претраге, умјесто да се у *Elasticsearch* копира цјелокупна структура података која већ постоји у релационој бази.

Приликом избора поља узета је у обзир и улога релационе базе као примарног извора података. Поља која нису неопходна за претрагу, као што су опис епизоде, трајање, број прегледа, број свиђања, путања до датотеке и интернет адреса (енг. *Uniform Resource Locator, URL*), нису дуплирана у *Elasticsearch*-у. Дескрипција видеа није укључена у поља предвиђена за претрагу, јер је ријеч о метаподатку који не представља транскрибовани садржај епизоде, а његово укључивање, ради дужине, могло би утицати на релевантност резултата претраге. Посебно је значајно што се вриједности попут броја прегледа и броја свиђања могу мијењати током времена, па њихово задржавање у *PostgreSQL*-у избјегава непотребно синхронизовање између два система. Након претраге, *Elasticsearch* враћа идентификаторе релевантних видеа, на основу којих се остали подаци могу дохватити из релационе базе ради приказа на корисничком интерфејсу.

Са друге стране, у *Elasticsearch* су укључена поља која су директно повезана са претрагом и приказом резултата: идентификатор видеа (*video_id*), идентификатор сегмента (*segment_id*), садржај сегмента (*text*), наслов видеа (*title*), временски почетак сегмента (*start*), временски крај сегмента (*end*), наслов канала (*channel_title*) и вријеме објаве видеа (*published_at*). Поље *text*, представник садржаја сегмента, дефинисано је кроз тип податка „*text*“ и представља основно поље за претрагу транскрипта, док је наслов видеа (*title*), такође дефинисан као тип податка „*text*“ како би корисник могао претраживати и наслове епизода. Идентификатор видеа (*video_id*) је дефинисан као тип податка „*keyword*“, како би се користи за идентификацију и повезивање резултата са подацима у релационој бази података. Идентификатор сегмента (*segment_id*) служи за идентификацију и очување редослиједа сегмената, док временски почетак сегмента (*start*) и временски крај сегмента (*end*) омогућавају враћање тачног временског интервала унутар видеа. Поља наслов канала (енг. *channel_title*) и вријеме објаве видеа (*published_at*) су укључена како би се омогућило филтрирање и сортирање резултата по каналу и датуму објављивања.

На овај начин *Elasticsearch* индекс садржи само податке релевантне за претрагу, филтрирање и идентификацију резултата, док *PostgreSQL* остаје централно мјесто за трајно чување комплетних података о епизодама. Оваква подјела одговорности смањује непотребно дуплирање података и омогућава да се промјенљиви подаци ажурирају у релационој бази без потребе за њиховим поновним индексирањем у *Elasticsearch*-у.

Мапирање се поставља у конфигурацији индекса. На тај начин *Elasticsearch* добија информацију о структури докумената и начину на који треба да обрађује њихова поља приликом претраге. Дио конфигурације којим је дефинисано мапирање коришћено у овој имплементацији приказан је у листингу 7.

```
"mappings": {
  "properties": {
    "video_id": {
      "type": "keyword"
    },
    "segment_id": {
      "type": "integer"
    },
    "text": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "title": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "start": {
      "type": "float"
    },
    "end": {
      "type": "float"
    },
    "channel_title": {
      "type": "keyword"
    },
    "published_at": {
      "type": "date"
    }
  }
}
```

Листинг 7: Конфигурација мапирања *Elasticsearch* индекса

6.8 Изазови српског језика у *Elasticsearch*-у

Примјена *Elasticsearch*-а над корпусом српског језика доноси неколико језички-специфичних изазова, детаљно обрађених у поглављу 2:

- богату флексију (падеже и глаголска времена — нпр. облици „Косово“, „Косову“, „Косова“ и „Косовом“). Са морфолошке стране гледано, ове ријечи представљају исти појам, али подразумевијевано подешавање их третира као различите термине.
- присуство два писма (ћирилица и латиница)
- дијакритике
- непостојање уграђене подршке за стематизацију и лематизацију српског језика у оквиру *Elasticsearch*-а.

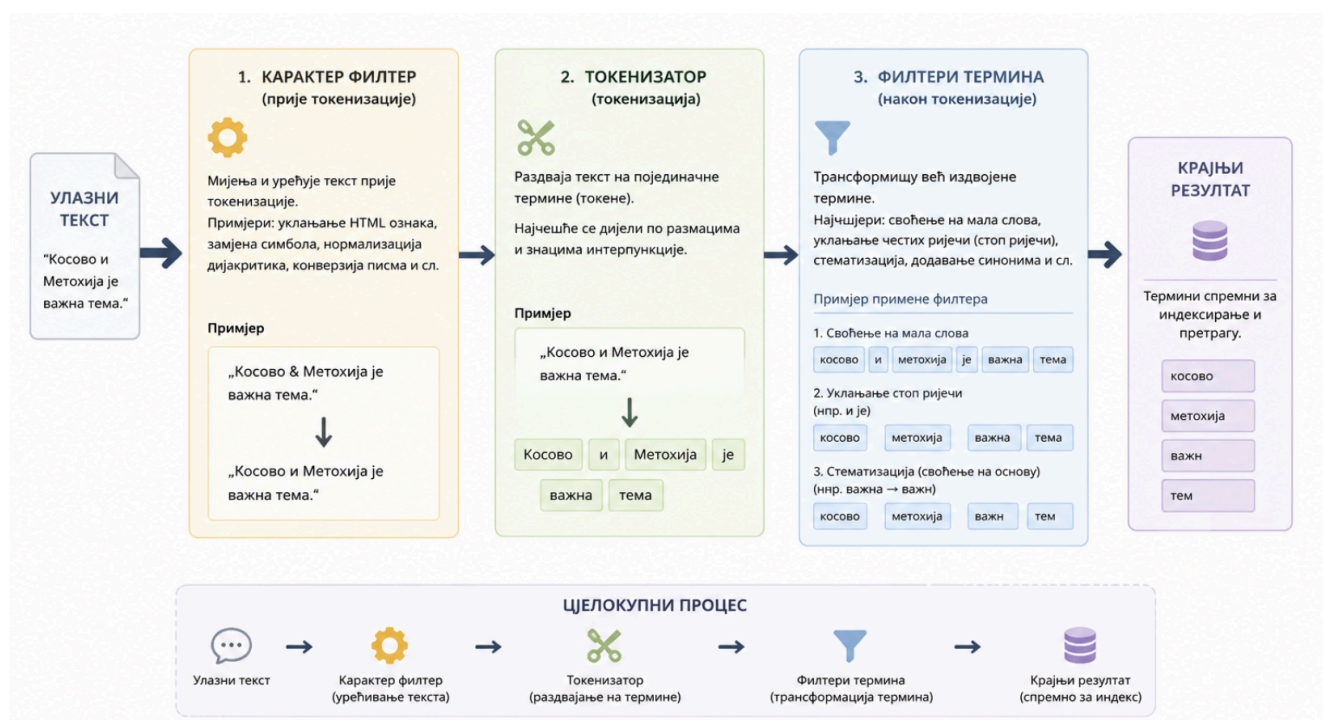
Ниједан од ових изазова није ријешен подразумијеваним (енг. *default*) подешавањем система, због чега је за потребе овог рада развијен прилагођени анализатор (енг. *analyzer*), описан у наставку овог поглавља.

6.9 Анализатор

Анализатор (енг. *analyzer*) представља ланац трансформација кроз који пролази текст прије него што буде записан у обрнути индекс (при индексирању) или упоређен са садржајем индекса (при претраживању) [23]. Сваки анализатор се састоји од три компоненте:

- карактер филтера (енг. *character filter*) - мијења текст прије токенизације;
- токенизатора (енг. *tokenizer*) - раздваја текст на појединачне термине;
- низа филтера термина (енг. *token filter*) - трансформише већ издвојене термине, нпр. свођење на мала слова, уклањање честих ријечи, стематизацију и сл. [23];

На слици 5 приказан је поједностављен ток анализе текста кроз наведене компоненте анализатора.



Слика 5: Поједностављен приказ процеса анализе текста у *Elasticsearch*-у

Elasticsearch подразумијевано нуди стандардни (енг. *standard*) анализатор, који представља опште рјешење за анализу текста на различитим језицима. Он раздваја текст на термине према границама ријечи, уклања већину знакова интерпункције и претвара термине у мала слова. Међутим, овакав приступ не узима у обзир специфичности појединачних језика, као што је њихова морфологија, зауставне ријечи (енг. *stop words*), те различите облике исте ријечи. Због тога стандардни анализатор није био довољан за потребе претраге

корпуса на српском језику, већ је дефинисан прилагођени ланац филтера који омогућава адекватнију обраду српског текста.

6.9.1 Прилагођени анализатор за српски језик

За потребе лексичке претраге коришћен је прилагођени анализатор за српски језик. *Elasticsearch* поред стандардног анализатора пружа и језички специфичан српски (енг. *serbian*) анализатор, који уз стандардну токенизацију подржава уклањање зауставних ријечи (енг. *stop words*) и стематизацију карактеристичну за српски језик. Његова конфигурација може се реализовати и као прилагођени анализатор, што омогућава додавање додатних корака у процес анализе [26].

У првом кораку примјењује се **icu_folding филтер** из *ICU (International Components for Unicode)* додатка, који обезбјеђује Unikod (енг. *Universal Character Encoding, Unicode*) нормализацију и пресавијање карактера [27], што је посебно значајно за рад са дијакритичким знаковима. *ICU* додатак се користи за рјешавање проблема два писма ћирилице и латинице, тако да упит „Србија“ и упит „Srbija“ погађају исти термин у индексу. Након тога се примјењује **филтер lowercase**, којим се сви термини свде на мала слова, чиме се избјегава разликовање истих ријечи на основу величине слова.

Сљедећи корак представља уклањање зауставних ријечи (енг. *stop words*). *Elasticsearch* обезбјеђује уграђену листу српских зауставних ријечи означену са **serbian_stop**, коју је могуће користити у прилагођеном анализатору [28]. У оквиру имплементације коришћена је ова листа, а њена примјена је посебно значајна за транскрипте, у којима се често појављују кратке функционалне ријечи које не доприносе значајно релевантности резултата. Примјер зауставних српских ријечи можете пронаћи у листингу 8 [29].

```
"ili", "a", "ali", "pa", "biti", "ne", "jesam", "sam", "jesi", "si",
"je", "jesmo", "smo", "jeste", "ste", "jesu", "su", "nijasam", "nisam", "nijas",
"nisi", "nije", "nijasmo", "nismo", "nijeste", "niste", "nijas", "nisu", "budem", "budeš",
"bude", "budemo", "budete", "budu", "budes", "bih", "bi", "bismo", "biste", "biše",
"bise", "bio", "bili", "budimo", "budite", "bila", "bilo", "bile", "ću", "ćeš",
"će", "ćemo", "ćete", "neću", "nećeš", "neće", "nećemo", "nećete", "cu", "ces",
"ce", "cemo", "cete", "necu", "neces", "nece", "necemo", "necete", "moгу",
"можеш", "може", "можемо", "можете", "mozes", "moze", "mozemo", "mozete", "и", "или",
"а", "али", "па", "бити", "не", "јесам", "сам", "јеси", "си", "је",
"јесмо", "смо", "јесте", "сте", "јесу", "су", "нијесам", "нисам", "нијеси",
"ниси", "није", "нијесмо", "нисмо", "нијесте", "нисте", "нијесу", "нису", "будем", "будеш",
"буде", "будемо", "будете", "буду", "будес", "бих", "би", "бисмо", "бисте",
"бише", "бисе", "био", "били", "будимо", "будите", "била", "било", "биле", "ћу",
"ћеш", "ће", "ћемо", "ћете", "нећу", "нећеш", "неће", "нећемо", "нећете", "цу",
"цес", "це", "цемо", "цете", "нецу", "нецес", "неце", "нецемо", "нецете", "могу",
"можеш", "може", "можемо", "можете", "mozес", "мозе", "можемо", "можете"
```

Листинг 8: Списак зауставних ријечи у уграђеној листи *serbian_stop*

За рјешавање морфолошке варијативности српског језика примјењује се **serbian_stemming**. *Elasticsearch* за српски језик обезбјеђује стемер који се може користити као **stemmer филтер** са параметром **language: "serbian"** [30]. На овај начин различити облици исте ријечи могу бити сведени на заједнички облик, чиме се повећава вјероватноћа да ће упит написан у једном граматичком облику пронаћи документе у којима се појављује други облик исте ријечи. На примјер, претрага по термину „Косово“ може обухватити и појављивања облика „Косову“ или „Косова“. Коначан ланац анализе коришћен за поља *text* и *title* приказан је у листингу 9.

```

"analysis": {
  "analyzer": {
    "serbian_analyzer": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": [
        "icu_folding",
        "lowercase",
        "serbian_stop",
        "serbian_stemming"
      ]
    }
  },
  "filter": {
    "serbian_stop": {
      "type": "stop",
      "stopwords": "_serbian_"
    },
    "serbian_stemming": {
      "type": "stemmer",
      "language": "serbian"
    }
  }
}

```

Листинг 9: Конфигурација прилагођеног анализатора за српски језик

6.9.2 Скраћеница: два анализатора, за индексирање и за претрагу

Ланац филтера описан у претходном одјелку не рјешава један специфичан проблем — скраћенице и акрониме. Корисник који упише „КиМ“ очекује резултате који садрже пуни израз „Косово и Метохија“, иако та два низа знакова немају ниједан заједнички термин: ниједна стематизација ни нормализација писма то не премошћава. Проблем се не рјешава ни семантичком (векторском) претрагом (Глава 7). Скраћенице представљају слијепу тачку (енг. *blind spot*) *embedding* модела, јер модел вјероватно није, или је ријетко, видео дати акроним током тренирања, те за њега не постоји поуздана векторска репрезентација, за разлику од појмовних синонима које векторски модел углавном препознаје и без експлицитне листе.

Рјешење је филтер за синониме (енг. *synonym token filter*), који при обради текста додаје алтернативне облике термина. Кључна одлука тиче се тренутка у ком се синоними примјењују: при индексирању (енг. *index time*) или при претраживању (енг. *search time*). Да су синоними примијењени при индексирању, свака измјена листе скраћеница захтијевала би поновно индексирање цијелог корпуса — у случају овог система, реиндексирање преко девет и по милиона докумената, у трајању преко 60 минута. С обзиром на то да се листа скраћеница очекивано мијења током даљег рада и тестирања система, одабрана је примјена искључиво при претраживању.

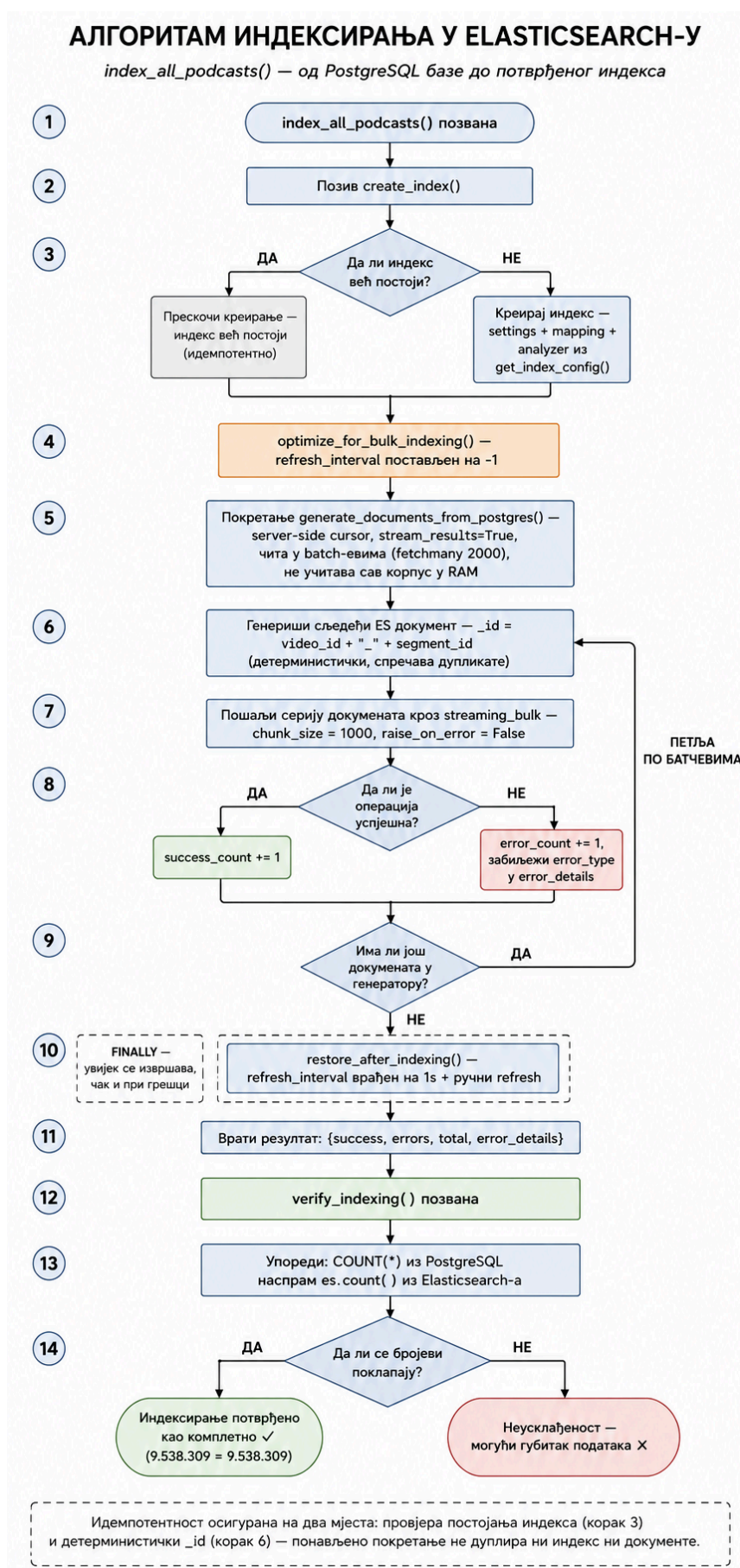
Ова одлука захтијева дефинисање два одвојена анализатора: индексни анализатор (*serbian_analyzer*), кориштен приликом индексирања и наведен директно у мапирању поља *text* и *title*, и претраживачки анализатор (*serbian_search_analyzer*), који садржи исти ланац филтера уз додатак филтера за синониме, а који се експлицитно наводи као параметар *analyzer* у самом упиту за претрагу, умјесто да буде дио мапирања. Захваљујући овом раздвајању, додавање нове скраћенице своди се на измјену подешавања индекса (затварање индекса, ажурирање подешавања, отварање индекса) — операцију која траје неколико секунди, без потребе за реиндексирањем докумената. Листа имплементираних скраћеница релевантних за корпус дата је у листингу 10.

```
SYNONYMS = [  
    "kim, kosovo i metohija, kosmet",  
    "bih, bosna i hercegovina",  
    "rs, republika srpska",  
    "cg, crna gora, montenegro",  
    "sad, sjedinjene americke drzave, amerika, usa",  
    "eu, evropska unija",  
    "nato, sjeverno-atlantski savez",  
    "un, ujedinjene nacije",  
    "mmf, medjunarodni monetarni fond",  
    "rts, radio televizija srbije",  
    "sns, srpska napredna stranka",  
    "sps, socijalisticka partija srbije",  
    "bdp, bruto domaci proizvod",  
]
```

Листинг 10: Списак скраћеница коришћених у *Elasticsearch* претрази

6.10 Алгоритам индексирања

Цјелокупан поступак индексирања — од провјере постојања индекса, преко генерисања и слања документа, до финалне верификације — приказан је као дијаграм тока на слици [6](#), а детаљно је образложен у наставку овог одјелка.



Слика 6: Алгоритам индексирања у *Elasticsearch*-у — од провјере постојања индекса до верификације учитаних докумената

6.10.1 Креирање индекса

Функција `create_index()` прије креирања провјерава да ли индекс са датим именом већ постоји (корак 3 на слици 6) — та провјера чини операцију идемпотентном: позивање функције више пута (нпр. приликом поновног покретања *backend* сервиса) неће обрисати нити дуплирати постојећи индекс. Уколико индекс не постоји, креира се позивом *Elasticsearch API*-ја са комплетном конфигурацијом добијеном из `get_index_config()` — подешавањима, мапирањем и *analyzer* ланцем описаним у претходним одјелцима.

6.10.2 Bulk индексирање

Индексирање преко девет и по милиона докумената захтјева другачији приступ од упућивања појединачног захтјева за сваки документ. У наставку је образложено зашто је коришћен *Bulk API* умјесто индексирања документ-по-документ. *Bulk API (bulk)* представља *Elasticsearch API endpoint* који омогућава извршавање више операција индексирања, креирања, брисања или ажурирања докумената у оквиру једног *HTTP* захтјева, чиме се смањује мрежни трошак и значајно повећава брзина индексирања [31]. Потом сљедују објашњења конкретне оптимизација и тока извршавања самог поступка (кораки 4 - 11 на слици 6).

6.10.2.1 Зашто *bulk*, а не документ-по-документ

Слање једног *HTTP* захтјева за сваки појединачни документ носи фиксни трошак (успостављање везе, парсирање захтјева, координација унутар кластера) који се, при обиму од преко девет и по милиона докумената, вишеструко умножава и чини индексирање непрактично спорим. *Elasticsearch* за то нуди *Bulk API* [31] — један *HTTP* захтјев који садржи низ операција индексирања, чиме се фиксни трошак по захтјеву амортизује на цијелу серију докумената умјесто на сваки појединачно.

Да би се читав корпус проследио *Bulk API*-ју без исцрпљивања *RAM* меморије, документи се не учитавају одједном у листу, него генеришу један по један помоћу функције `generate_documents_from_postgres()` (корак 5 на слици 6).

Идентификатор сваког документа се експлицитно поставља на `f"{video_id}_{segment_id}"` (корак 6), умјесто да га *Elasticsearch* генерише аутоматски. Овим се индексирање чини идемпотентним: уколико се поступак индексирања понови (нпр. после прекида или измјене у подацима), документ са истим `video_id` и `segment_id` је замјењен (*upsert*), а не дуплиран.

6.10.2.2 Оптимизација и покретање

Функција `optimize_for_bulk_indexing()` привремено искључује освјежавање индекса (`refresh_interval`), подешавајући га на `-1` за вријеме трајања *bulk* индексирања (корак 4 на слици 6). Подразумијевано, *Elasticsearch* освјежава индекс на сваку секунду како би нови документи одмах били доступни за претрагу — при индексирању милиона докумената у кратком временском периоду то освјежавање непотребно троши ресурсе, јер документи не морају бити тренутно претраживи док траје почетно пуњење индекса. По завршетку индексирања, `refresh_interval` се враћа на уобичајену вриједност (корак 10, извршен у *finally* блоку — гарантовано се извршава чак и уколико дође до грешке током индексирања). Ова оптимизација одговара званичној *Elastic* препоруци за убрзавање индексирања при почетном пуњењу великих индекса [32].

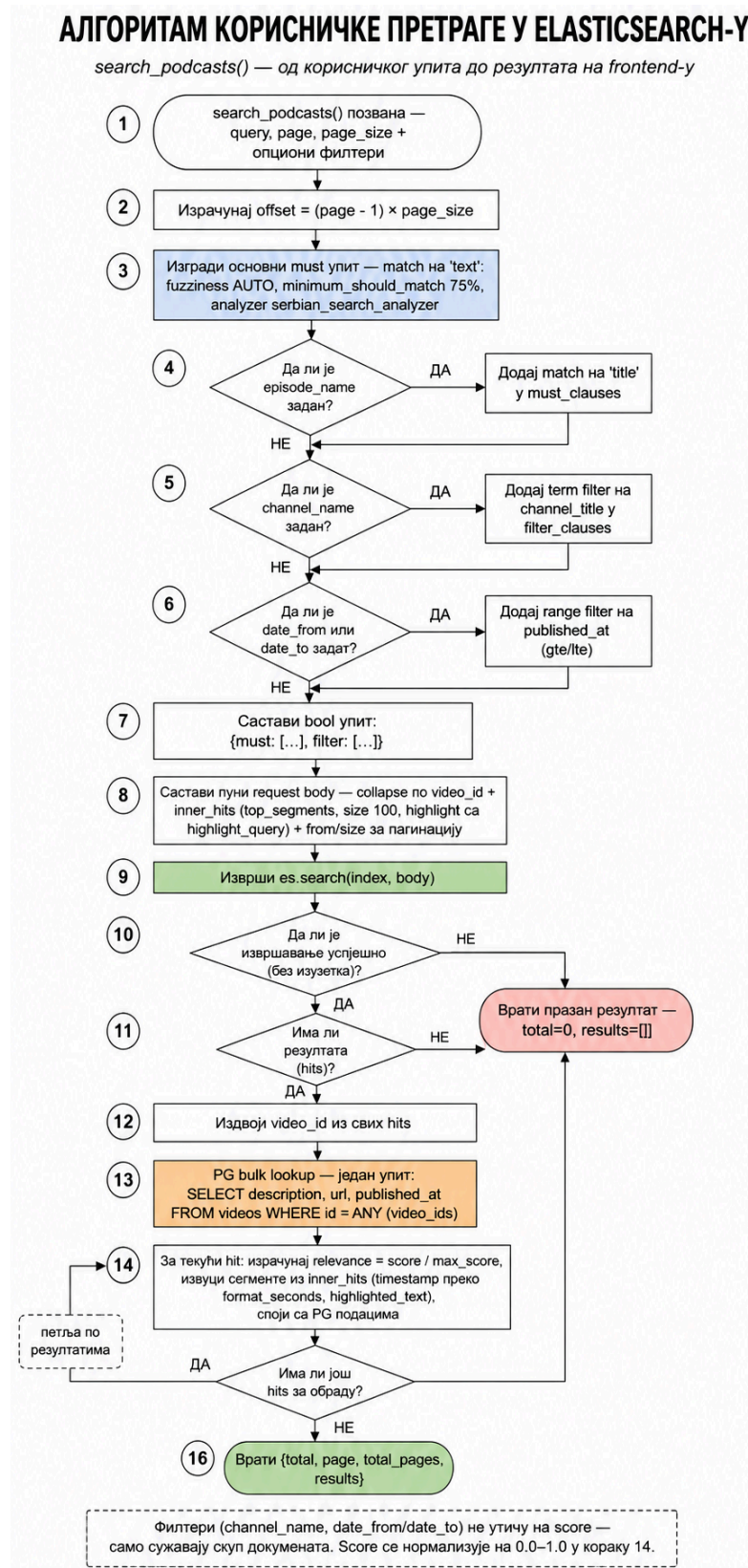
Поступак је покретан преко административног *endpoint*-а (`POST /admin/index/create`), након што су провјерени `/health` статуси базе и *Elasticsearch* кластера. Резултат покретања над цјелокупним корпусом дат је у листингу 11.

```
{"success": 9538309, "errors": 0, "total": 9538309}
```

Листинг 11: Резултат *bulk* индексирања докумената

6.11 Алгоритам претраге

Док претходни одјељак описује једнократни поступак попуњавања индекса, овдје је приказан ток који се извршава при свакој корисничкој претрази — од примљеног упита, преко грађења *Elasticsearch* упита, до формирања одговора за корисничког интерфејса. Комплетан ток приказан је на слици 7.



Слика 7: Алгоритам корисничке претраге у *Elasticsearch-у*

6.11.1 Грађење *Elasticsearch* упита

Функција `search_podcasts()` прима текстуални упит (енг. *query*) као обавезан параметар, те четири опциона филтера — назив епизоде, назив канала и опсег датума — уз параметре страничења *page* и *page_size* (корак 1 на слици 7).

Основни дио упита је увијек исти: ***match*** над пољем ***text***, при чему ***fuzziness***: “**AUTO**” омогућава толерирање мањих правописних грешака (*minimum_should_match*: “75%”), а *analyzer*: “*serbian_search_analyzer*” обезбјеђује анализу прилагођену српском језику. Упит ***match*** представља стандардни начин за пуну текстуалну претрагу у *Elasticsearch*-у [33]. Основни дио упита приказан је у листингу 12.

```
must_clauses = [
    {
        "match": {
            "text": {
                "query":          query,
                "fuzziness":      "AUTO",
                "minimum_should_match": "75%",
                "analyzer":      "serbian_search_analyzer"
            }
        }
    }
]
```

Листинг 12: Конфигурација *match* упита за претрагу текста у *Elasticsearch*-у

Три опциона услова додају се у упит само уколико их је корисник заиста задао (кораци 4–6). Назив епизоде се додаје као додатни ***match*** у ***must_clauses***, док назив канала и опсег датума иду у ***filter_clauses***. Параметри у ***must*** дијелу упита утичу на *BM25* скор релевантности, док параметри у ***filter*** дијелу само искључују документе из резултата, без утицаја на рангирање.

6.11.2 Collapse, inner_hits и highlight

Прије слања упита, *query* се уграђује у комплетан ***request body*** (корак 8). Два додатна механизма овдје су кључна за понашање претраге:

- ***collapse*** по пољу ***video_id*** осигурава да се сваки видео у резултатима прикаже само једном. За сваки видео приказује се један главни резултат, док се до 100 најрелевантнијих сегмената тог видеа издваја посебно, у оквиру ***inner_hits***.
- ***highlight*** конфигурација означава подударачуће дијелове текста ***mark*** ознакама.

6.11.3 Извршавање, *PG bulk lookup* и формирање одговора

Позив `es.search()` (корак 9 на слици 7) обавијен је ***try/except*** блоком — уколико упит из некаквог разлога не успије, враћа се празан резултат (***total: 0, results: []***) умјесто да цијела претрага откаже (корак 10). Исти празан резултат враћа се и уколико упит успије, али не пронађе ниједан одговарајући документ (корак 11).

Уколико резултата има, ***video_id*** вриједности свих враћених документа издвајају се у јединствену листу (корак 12), над којом се извршава један упит ка *PostgreSQL* бази — исти принцип *bulk_lookup*-а који је већ примјењен приликом индексирања. Овдје га примјењујемо у супротном смјеру: један упит за опис, интернет адресу и датум објављивања свих видеа из резултата, умјесто једног упита по видеу (кораци 12–13), приказ исјечка кода кроз листинг 13.


```

video_ids = [hit["_source"]["video_id"] for hit in hits]

result = conn.execute(
    text("""
        SELECT id, description, url, published_at
        FROM videos
        WHERE id = ANY(:ids)
    """),
    {"ids": video_ids}
)
pg_data = {row.id: row for row in result}

```

Листинг 13: Дохватање метаподатака релационе базе

За сваки резултат затим се рачуна нормализован износ релевантности, дијељењем сировог (енг. raw) *ES* износа максималним износом у скупу резултата (***round (hit_score / max_score, 2)***), а временске ознаке сегмената претварају се из секунди у читљив формат (***format_seconds()***, нпр. *83.4* → ***“1:23”***) — ова петља понавља се за сваки резултат (кораци 14–15), приказан исјечак кода у листингу 14.

```

relevance = round((hit["_score"] / max_score), 2)

matches = []
for seg_hit in hit["inner_hits"]["top_segments"]["hits"]["hits"]:
    seg = seg_hit["_source"]
    highlighted = seg_hit.get("highlight", {}).get(
        "text", [seg["text"]]
    )[0]

    matches.append({
        "timestamp": format_seconds(seg["start"]),
        "timestamp_seconds": int(seg["start"]),
        "text": seg["text"],
        "highlighted_text": highlighted
    })

```

Листинг 14: Нормализација скора релевантности и форматирање временских ознака сегмената

На крају, *ES* подаци (наслов, канал, сегменти, релевантност) спајају се са *PostgreSQL* подацима (опис, интернет адреса, датум објаве) у јединствен одговор који се просљеђује функцији хибридизације (поглавље 8).

Семантичка (векторска) претрага

Семантичка (енг. *semantic search*), односно векторска претрага (енг. *vector search*) је приступ претраживању текста код кога се, умјесто поређења низова карактера, пореди значење текста [34]. Упит и садржај документа представљају се у облику нумеричких вектора, а њихова сличност мјери се удаљеношћу или углом између тих вектора у вишедимензионалном простору. Два текста која изражавају сличан садржај добијају блиске векторе без обзира на то да ли дијеле исте ријечи, чиме векторска претрага директно допуњује лексичку претрагу описану у поглављу 6. У оквиру овог рада, семантичка претрага имплементирана је коришћењем *embedding* модела за генерисање векторских репрезентација и *pgvector* екстензије *PostgreSQL* базе података за њихово складиштење и претрагу по сличности.

Потреба за оваквим приступом директно произлази из ограничења лексичке претраге, која су дјелимично већ описана у поглављу 2.4. Проблем није ограничен на појединачне, унапријед познате парове синонима, попут „додаци исхрани“ и „суплементи“, јер се такви случајеви, у принципу, могу ријешити ручно одржаваном листом синонима, слично начину на који су скраћенице попут „КиМ“ ријешене у *Elasticsearch*-у (поглавље 6.9.2). Међутим, проблем је много шире природе и у литератури информационог претраживања познат је под називом вокабуларни јаз (енг. *vocabulary problem*). Прва систематска емпиријска студија овог феномена показала је да двоје различитих људи, независно једно од другог, за исти појам или радњу бирају исти термин у мање од 20% случајева [35]. Корисник система описаног у овом раду и гост подкаста управо су двоје различитих људи, па је вјероватноћа да ће корисников упит садржати управо оне ријечи које је гост изговорио, нарочито код дужих и описних упита, релативно мала. На примјер, за упит „како узгајати медитеранску културу“, транскрипт у којем се говори о томе да „маслине траже доста сунца“ и да је „смоква захвална биљка“ не садржи ниједан заједнички термин са упитом, иако је њихова тематска повезаност очигледна.

Још један могући проблем за лексичку претрагу биле би грешке аутоматске транскрипције, односно шума. Уколико *ACR* модел, односно технологија аутоматског препознавања говора, (поглавље 5.2) погрешно транскрибује термин „дречавци“ као два одвојена токена, лексичка претрага неће пронаћи резултат за упит „дречавци“ у тој епизоди, јер термин у облику који упит очекује једноставно не постоји у индексу, без обзира на то колико прецизно *BM25* рангира резултате. Најзад, лексичка претрага не разликује значење истог термина у различитим контекстима, односно не разрјешава полисемију. Упит „култура“ подједнако би могао да врати епизоду о медитеранској пољопривреди, епизоду о култури сјећања и епизоду о бактеријској култури у лабораторији, јер је за *BM25* термин идентичан у сва три случаја, без обзира на околне ријечи које јасно одређују његово значење у датом контексту.

7.1 Векторске репрезентације текста

Векторска репрезентација текста (енг. *embedding*) представља пресликавање текста у вектор у вишедимензионалном простору. Циљ је да текстови сличног значења имају блиске векторске репрезентације. Према начину представљања, вектори могу бити ријетки (енг. *sparse*) и густе (енг. *dense*). Код ријетких репрезентација већина координата има вриједност нула, док су код густих репрезентација координате углавном различите од нуле. За разлику од лексичких репрезентација, које се заснивају на присуству и учесталости термина, научени *embedding* настоји да у векторском простору представи и семантичке односе између текстова [36].

У оквиру овог система коришћене су *dense* репрезентације текста. Модел пресликава упит и сегменте транскрипта у векторе фиксне димензије, након чега се њихова сличност користи за проналажење семан-

тички релевантних садржаја. На тај начин систем може да препознаје сличност и између текстова који не користе исте термине, али изражавају слично значење.

Кључно својство простора у којем се *embedding* вектори налазе јесте да позиција вектора зависи од контекста у којем се термин или реченица јављају, а не само од самог термина [34]. Управо ово својство објашњава зашто *embedding* модел рјешава проблем полисемије поменут у претходном одјељку: термин „култура“ у реченицама о медитеранској пољопривреди биће пресликан у другу регију вишедимензионог простора него исти термин у реченицама о култури сјећања, јер модел приликом генерисања вектора узима у обзир и околне ријечи.

Термин *embedding* у литератури се користи у два повезана, али различита значења, која је корисно раздвојити ради јасноће терминологије коришћене у наставку овог поглавља [34]. У једном значењу, *embedding* означава сам поступак пресликавања података у вектор. У другом значењу, *embedding* означава резултат тог поступка — конкретан вектор који из њега произилази, или цијелу колекцију таквих вектора. Из контекста реченице у наставку текста увијек је јасно на које се значење мисли.

7.2 Мјера сличности: *dot product*, *cosine similarity* и еуклидско растојање

Сваки вектор посједује два својства: дужину (норму, $\|A\| = \sqrt{\sum_i a_i^2}$) и смјер. Код текстуалних *embedding* вектора, смјер код многих модела носи информацију о семантичкој сличности, док норма може бити повезана са другим својствима репрезентованог текста, као што су његова дужина или количина садржаја [34]. Ова разлика представља један од фактора који утичу на избор мјере сличности.

Најједноставнији начин поређења два вектора јесте унутрашњи производ (енг. *dot product*), чија геометријска интерпретација гласи:

$$q \cdot u = \|q\| \cdot \|u\| \cdot \cos(\theta)$$

Унутрашњи производ, дакле, зависи од два одвојена својства — сличности смјера, одређене углом θ , и производа дужина оба вектора [36]. То значи да вектор веће норме може остварити већи унутрашњи производ чак и када је његов смјер исти као код вектора мање норме. У контексту претраге, то може довести до тога да дужи текстови добијају предност у рангирању када је норма њихових *embedding* вектора повезана са дужином или количином текста [34].

Косинусна сличност (енг. *cosine similarity*) ублажава овај утицај тако што унутрашњи производ нормализује дужинама оба вектора:

$$\cos(q, u) = \frac{q \cdot u}{\|q\|_2 \|u\|_2}$$

На тај начин норма вектора не утиче на вриједност мјере, већ резултат зависи од угла између вектора, односно њиховог смјера [36]. Косинусна сличност узима вриједности у интервалу $[-1, 1]$, при чему 1 означава исти смјер вектора, а 0 њихову ортогоналност. У контексту текстуалних *embedding*-а, већа косинусна сличност обично указује на већу семантичку сличност, у зависности од својстава конкретног *embedding* модела [34].

Ово својство је посебно релевантно за претрагу у којој се пореде текстови различите дужине. На примјер, кратак корисников упит, који се може састојати од једне или двије ријечи, и дужи сегмент транскрипта, који садржи неколико реченица, могу имати *embedding* векторе различитих норми. Уколико би норма била повезана са дужином текста, коришћење унутрашњег производа могло би фаворизовати дуже текстове. Косинусна сличност уклања тај утицај нормализацијом вектора и тиме омогућава да се поређење заснива првенствено на њиховом смјеру [34].

Трећа често коришћена мјера јесте еуклидско растојање (енг. *Euclidean distance*, L_2), које мјери удаљеност између два вектора у простору:

$$d(q, u) = \text{sqrt} \left\{ \sum_i (q_i - u_i)^2 \right\}$$

[36]

За разлику од претходне двије мјере, гдје већа вриједност означава већу сличност, код еуклидског растојања важи обрнуто — мања вриједност означава већу сличност. Еуклидско растојање зависи и од норми вектора, а не само од њиховог смјера. То се може видјети из односа:

$$\|q - u\|_2^2 = \|q\|_2^2 - 2(q \cdot u) + \|u\|_2^2$$

[36]

Због тога два вектора који имају исти или веома сличан смјер, али различите дужине, могу бити међусобно удаљени према еуклидском растојању, иако указују на сличан семантички садржај. Из тог разлога, еуклидско растојање није коришћено у овом систему.

7.3 Нормализација: привођење свих мјера на исти поредак

Нормализација вектора подразумијева свођење његове дужине на јединичну вриједност, при чему смјер вектора остаје непромијењен [34]. Када су вектори нормализовани, унутрашњи производ постаје еквивалентан косинусној сличности, јер дужина вектора више не утиче на резултат поређења [34], [36]. Над истим векторима и еуклидско растојање даје исти поредак резултата као косинусна сличност, иако се њихове бројчане вриједности разликују [36].

Ово својство је искоришћено и у систему описаном у овом раду. *Embedding* вектори сегмената нормализују се при индексирању, а упитни вектор при свакој претрази. Након нормализације, сличност се може израчунати једноставним унутрашњим производом, који у том случају одговара косинусној сличности [34]. На тај начин поређење зависи од смјера вектора, а не од њихове дужине.

Овакав приступ одговара и начину употребе модела **multilingual-e5** коришћеног у овом раду, описаног у дијелу 7.6. За претрагу се *embedding* вектори нормализују, након чега се њихова сличност може израчунати унутрашњим производом. Над нормализованим векторима тај резултат одговара косинусној сличности. Због тога је исти приступ примијењен и у овом систему.

7.4 Проблем приближне претраге и HNSW

Проналажење (к) вектора из колекције који су најсличнији датом упитном вектору назива се **top-k** претрагом [36]. Код мањих колекција могуће је упоредити упит са сваким вектором и затим изабрати најсличније резултате. Међутим, са растом броја вектора овакав приступ постаје све скупљи, јер је за сваки упит потребно проћи кроз цијелу колекцију [36]. Због тога се код великих колекција користе алгоритми приближне претраге најближих сусјед (енг. *Approximate Nearest Neighbor*, ANN), који омогућавају знатно бржу претрагу уз могућност да резултат не садржи увијек апсолутно најближе векторе.

За приближну претрагу у овом систему коришћен је **Hierarchical Navigable Small World (HNSW)** индекс, који подржава **pgvector**. *HNSW* организује векторе у облику вишеслојног графа, при чему везе између чворова омогућавају ефикасно кретање према векторима који су најближи упиту [37].

Претрага почиње на вишим слојевима графа, који садрже мањи број чворова и омогућавају брзо кретање кроз простор. Након проналажења одговарајуће области, претрага се наставља на нижим слојевима, све до основног слоја који садржи све векторе. На тај начин није потребно поредити упит са сваким вектором у колекцији, што *HNSW* чини погодним за претрагу великих колекција *embedding* вектора [37].

7.5 *pgvector* као векторска база података

Складиштење и претрага *embedding* вектора у овом систему реализовани су помоћу *pgvector* екстензије за *PostgreSQL*, чија је основна конфигурација већ описана у поглављу 4.1. Приликом избора технологије разматране су и алтернативе — *FAISS*, *Qdrant*, *Milvus* и *Pinecone*.

FAISS је библиотека за претрагу најближих сусједа, али није самостална база података, па не обезбјеђује трајно складиштење података нити управљање метаподацима. Са друге стране, *Qdrant* и *Milvus* су намјенске векторске базе података, првенствено намијењене веома великим колекцијама вектора и дистрибуираним системима, док је *Pinecone* управљана услуга у облаку која подразумева спољну инфраструктуру и текуће трошкове коришћења [34].

За потребе овог рада, *pgvector* се показао као најприкладније рјешење, јер омогућава да иста *PostgreSQL* база која већ чува метаподатке служи и за складиштење *embedding* вектора. На тај начин није било потребно уводити засебну векторску базу у архитектуру система [34].

Додатна предност *pgvector*-а је могућност комбиновања векторске претраге са стандардним *SQL* упитима, укључујући филтрирање по метаподацима као што су канал или датум објављивања. Иако ова могућност није искоришћена у тренутној имплементацији, него се филтрација користи накнадно у хибридизацији, она оставља простор за будућа проширења система.

7.6 Избор *embedding* модела

Квалитет семантичке претраге у великој мјери зависи од избора *embedding* модела, који текст претвара у векторску репрезентацију. За потребе овог система разматрана су три модела:

- *multilingual-e5-base*;
- *multilingual-e5-large* [38];
- *bge-m3* [39];

bge-m3 подржава три начина претраге: *dense*, *sparse* и *multi-vector* претрагу [39]. Пошто је лексичка претрага у овом систему већ реализована помоћу *Elasticsearch*-а, за семантичку компоненту био је довољан модел намијењен *dense* претрази. Због тога додатне могућности модела *bge-m3* нису биле неопходне, па је предност дата једноставнијем рјешењу заснованом на моделима из *E5* породице.

Између модела *multilingual-e5-base* и *multilingual-e5-large* направљен је компромис између квалитета претраге и потребних ресурса. На *MIRACL* тесту за вишејезичку претрагу, *multilingual-e5-base* остварује просјечни *nDCG@10* од 62,3, а *multilingual-e5-large* 66,5 [38]. *nDCG* (енг. *Normalized Discounted Cumulative Gain*) је мјера квалитета рангирања резултата претраге. Иако већи модел остварује бољи резултат, мањи модели захтијевају мање простора и омогућавају брже извршавање [38]. С обзиром на то да је систем развијан и тестиран у локалном окружењу са ограниченим ресурсима, изабран је *multilingual-e5-base*. Овај модел генерише векторе од **768 димензија**, што одговара **Vector(768)** колони дефинисаној у моделу података (поглавље 5.5).

7.7 Текстуални префикси

E5 модели при претрази разликују упит од текста који се претражује употребом префикса „*query*: “ и „*passage*: “. За задатке асиметричне претраге, као што је проналажење релевантног пасуса на основу корисничког упита, „*query*: “ додаје се упиту, а „*passage*: “ тексту који се претражује.

Ови префикси су важни за правилну употребу модела: њихово изостављање не спречава генерисање вектора, али може довести до смањења квалитета претраге. Због тога се у овом систему префикс „*passage*: “ додаје сваком чанку (енг. *chunk*) прије генерисања *embedding* вектора у **offline pipeline**-у (одјелјак 7.10), док се „*query*: “ додаје корисничком упиту прије генерисања вектора у **online pipeline**-у (одјелјак 7.11).

7.8 Грануларност и *chunking*

У поглављу 6.6 описана је грануларност лексичке претраге на нивоу транскрипцијских сегмената. За семантичку претрагу ти сегменти се додатно групишу у веће текстуалне јединице — ***chunk***-ове — које се затим претварају у ***embedding*** векторе.

7.8.1 Величина *chunk*-а

Величина *chunk*-а представља компромис између очувања контекста и прецизности претраге. Сувише мали *chunk*-ови могу изгубити важан контекст, док сувише велики могу умањити релевантност појединачног резултата [34]. Због тога се у овом систему не *embed*-ује ни цијела епизода као једна цјелина, нити сваки кратки транскрипцијски сегмент засебно, већ се сусједни сегменти групишу у веће цјелине.

Приликом избора стратегије разматрани су приступи засновани на фиксној величини текста, преклапању сусједних *chunk*-ова и семантичком одређивању граница. Фиксно сјечење је једноставно, али може раздвојити повезани садржај на граници *chunk*-ова, док преклапање омогућава задржавање контекста између сусједних *chunk*-ова. Семантички приступи настоје да повезани садржај задрже унутар истог *chunk*-а, али захтијевају сложенију обраду. У овом раду изабран је приступ који поштује постојеће границе транскрипцијских сегмената и користи преклапање између сусједних *chunk*-ова. Очување природних граница текста и употреба преклапања представљају уобичајене поступке при формирању *chunk*-ова за семантичку претрагу [34].

7.8.2 Алгоритам груписања сегмената

Функција ***build_chunks_for_video()*** (на путањи ***services/chunking_service.py***) редом групише транскрипцијске сегменте једне епизоде све док се не достигне задати максимални број токена. У имплементацији је постављено ***DEFAULT_MAX_TOKENS = 500***.

Дужина *chunk*-а мјери се у токенима јер *embedding* модел обрађује текст након токенизације и има ограничену максималну дужину улаза. Модел ***multilingual-e5-base*** подржава улаз дужине до 512 токена, при чему се дужи текст скраћује [40]. Због тога је у имплементацији постављена граница од 500 токена, чиме се оставља мала резерва у односу на максималну дужину модела, укључујући префикс ***“passage: “*** који се додаје тексту прије генерисања *embedding*-а.

При формирању *chunk*-а текстови сегмената спајају се редослиједом којим се појављују у транскрипту, док се почетно и крајње вријеме преузимају од првог, односно посљедњег сегмента. Између два сусједна *chunk*-а задржавају се посљедња два сегмента претходног *chunk*-а (***DEFAULT_OVERLAP_SEGMENTS = 2***). Преклапање се, дакле, дефинише на нивоу цијелих транскрипцијских сегмената, а не произвољног броја токена. На тај начин дио контекста са границе једног *chunk*-а остаје присутан и у сљедећем.

7.8.3 Гранични случајеви

Уколико један транскрипцијски сегмент сам премашује дозвољени број токена, он се додатно дијели на мање дијелове. Пошто база не чува временску ознаку за сваку појединачну ријеч, времена новонасталих дијелова процјењују се унутар интервала оригиналног сегмента.

Након формирања *chunk*-а број токена поново се израчунава над коначним спојеним текстом и та вриједност се уписује у *token_count*. На тај начин се чува стварни број токена текста који ће бити прослијеђен *embedding* моделу.

У случају да би пренесени дио из претходног *chunk*-а сам довео до прекорачења максималне величине, преклапање се за тај *chunk* изоставља. На тај начин ограничење величине има предност над преклапањем.

7.9 Припрема базе — HNSW индекс

Табела *Chunk*, укључујући колону *embedding* типа *Vector(768)*, дефинисана је у моделу података описаном у поглављу 5.5, док су *pgvector* екстензија и конфигурација *PostgreSQL* сервиса представљене у поглављу 4.1.

За ефикасну приближну претрагу над сачуваним *embedding* векторима додатно је креиран *HNSW* индекс, чији је принцип рада објашњен у одјелку 7.4.

pgvector омогућава креирање *HNSW* индекса директно над векторском колоном, при чему избор операторске класе зависи од мјере удаљености која се користи приликом претраге [15]. Пошто је за овај систем изабрана косинусна удаљеност, описана у одјелку 7.2, индекс је дефинисан помоћу операторске класе *vector_cosine_ops*, приказано у листингу 15.

```
CREATE INDEX ix_chunks_embedding_hnsw
ON chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

Листинг 15: Креирање *HNSW* индекса над *embedding* векторима

За параметре *m* и *ef_construction* задржане су подразумеване вриједности *pgvector*-а: *m* = 16 и *ef_construction* = 64 [15]. Параметар *m* одређује максималан број веза са сусједним чворовима по слоју *HNSW* графа, док *ef_construction* одређује величину листе кандидата која се разматра током изградње индекса. Повећање ових вриједности може побољшати квалитет претраге, али уз већу потрошњу меморије и дуже вријеме изградње индекса [15]. У овом раду параметри нису додатно оптимизовани за конкретан корпус, што оставља простор за будућу евалуацију и подешавање.

Креирање *pgvector* екстензије и *HNSW* индекса укључено је у *Alembic* миграцију базе. Пошто ови елементи захтијевају специфичне *PostgreSQL* наредбе, екстензија се креира директним извршавањем SQL наредбе у листингу 16. На тај начин *Alembic* остаје једини механизам за управљање шемом базе.

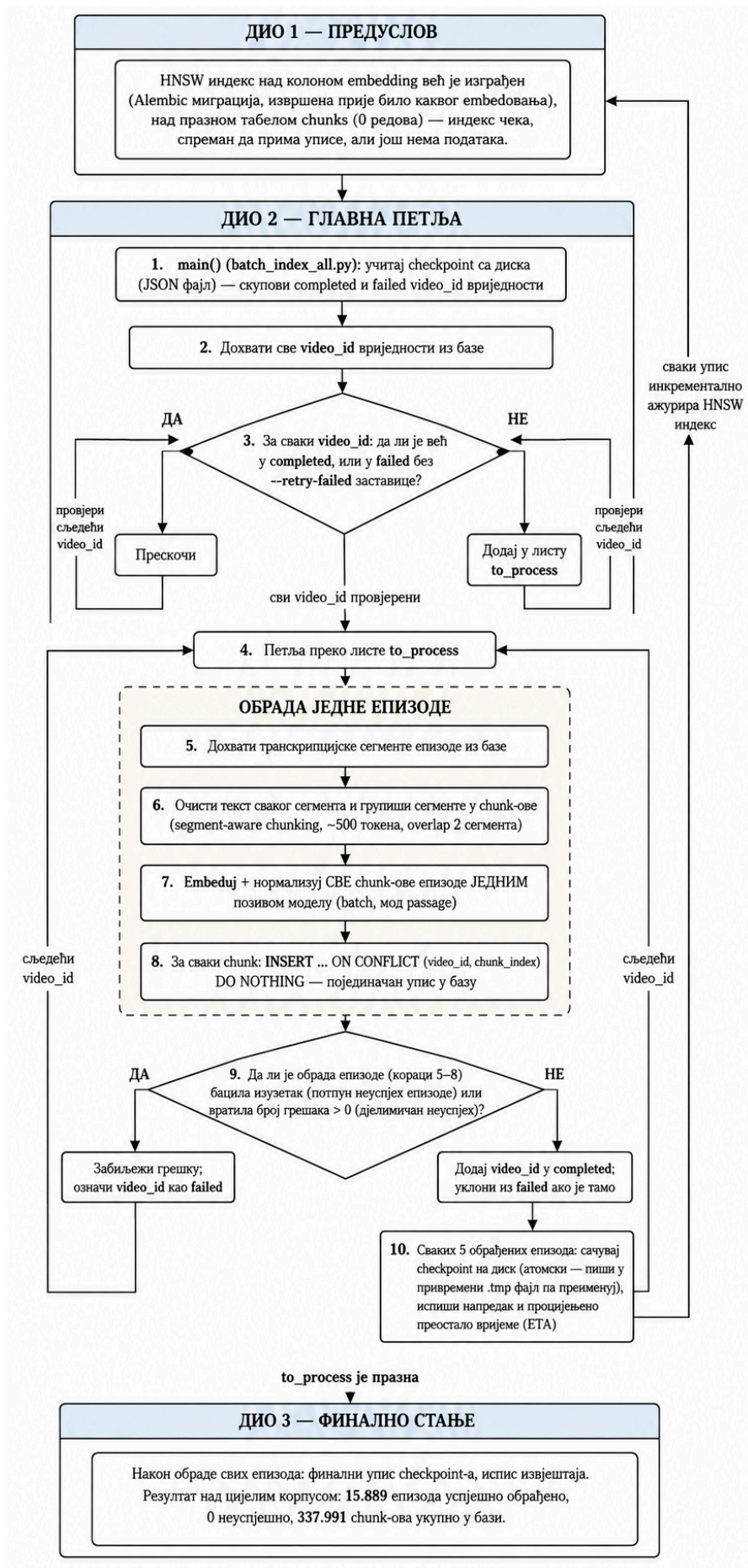
```
op.execute("CREATE EXTENSION IF NOT EXISTS vector;")
```

Листинг 16: Креирање *pgvector* екстензије у *Alembic* миграцији

7.10 Алгоритам индексирања (*offline pipeline*)

Генерисање *embedding* вектора за корпус обавља се у оквиру *offline pipeline*-а, прије него што систем постане доступан за претрагу. За разлику од овог поступка, *online pipeline* покреће се при сваком корисничком упиту и описан је у одјелку 7.11.

У наставку је најприје описана припрема и чишћење текста прије формирања *chunk*-ова, а затим поступак генерисања и уписа *embedding* вектора у базу за читав корпус. Комплетан ток овог процеса приказан је на слици 8.



Слика 8: Алгоритам batch embed-овања корпуса и уписа векторских репрезентација у базу

7.10.1 Чишћење текста прије *chunking*a

Прије подјеле транскрипта на *chunk*-ове, сваки транскрипцијски сегмент пролази кроз функцију `clean_segment_text()`. Чишћење се врши на нивоу појединачног сегмента како би се сачувала веза између текста и његових временских ознака (*start* и *end*).

Поступак обухвата *Unicode NFC* нормализацију, којом се различити *Unicode* записи истих карактера своде на јединствен облик, и транслитерацију, којом се текст написан ћирилицом преводи у одговарајући латинични запис. Након тога уклањају се *emoji* карактери, ознаке не-говорних звукова (нпр. „[музика]“ и „(смјех)“), сувишни симболи и размаци, као и типичне *Whisper* халуцинације, попут понављања истог текста и фраза „Хвала на гледању“ или „Претплатите се на канал“.

Понављања која се јављају унутар једног сегмента уклањају се током чишћења, док се понављања кроз више узастопних сегмената обрађују непосредно прије *chunking*-а. На тај начин се задржава само једно појављивање текста, уз одговарајућу временску ознаку.

7.10.2 Оркестрација поступка

Прије почетка *batch embed*-овања, *HNSW* индекс над колоном *embedding* већ је креиран *Alembic* миграцијом над празном табелом *chunks*. Током обраде корпуса индекс се затим аутоматски допуњује приликом уписа нових вектора.

Комплетан поступак реализован је скриптом `batch_index_all.py`. За сваку епизоду скрипта дохвата транскрипцијске сегменте из базе, чисти их и групише у *chunk*-ове, генерише *embedding* векторе за све *chunk*-ове епизоде једним позивом функције `embed_texts()` у режиму „*passage*“, а затим их уписује у базу.

За разлику од *Elasticsearch* индексирања, *batch embed*-овање покреће се као *CLI* скрипта приказана у листингу 17. Овакав приступ је изабран јер обрада цијелог корпуса траје приближно 20 сати, па дуготрајан *batch* процес није погодан за извршавање кроз синхрони *HTTP* захтев.

```
docker compose exec backend python batch_index_all.py
```

Листинг 17: Покретање *batch embed*-овања корпуса унутар *backend* контејнера

7.10.3 Checkpoint механизам и идемпотентност

Пошто обрада корпуса траје више сати, скрипта користи *checkpoint* датотеку `batch_index_checkpoint.json`, која омогућава наставак рада након прекида. У њој се чувају ***video_id*** вриједности успјешно обрађених (енг. *completed*) и неуспјешних (енг. *failed*) епизода. Већ завршене епизоде се при наредном покретању прескачу, док се неуспјешне могу поново обрадити употребом заставице ***-retry-failed***. *Checkpoint* се чува на сваких пет обрађених епизода.

Додатна заштита од дуплирања постоји на нивоу *chunk*-а. Упис се врши помоћу *sql* клаузуле приказане у листингу 18. Тако се *chunk* са истим ***video_id***-јем и ***chunk_index***-ом не може поново уписати у базу. Комбинација *checkpoint* механизма и ове провјере омогућава безбједан наставак *batch* обраде након прекида.

```
ON CONFLICT (video_id, chunk_index) DO NOTHING
```

Листинг 18: Спречавање дуплираног уписа *chunk*-ова у базу

7.10.4 Batch embed-овање и упис у базу

Embedding вектори генеришу се за све *chunk*-ове једне епизоде одједном, једним позивом модела. Ово је ефикасније од засебног позива модела за сваки *chunk*. Након генерисања вектора, *chunk*-ови се уписују појединачно у базу. Такав приступ омогућава да већ успјешно уписани *chunk*-ови остану сачувани чак и ако касније током обраде исте епизоде дође до грешке.

7.10.5 Руковање грешкама и завршни извјештај

Свака епизода обрађује се независно. Ако током њене обраде дође до грешке, епизода се означава као неуспјешна, а *batch* наставља са сљедећом. На тај начин грешка у једној епизоди не прекида обраду цијелог

корпуса. По завршетку скрипта исписује сажет извјештај о броју обрађених, успјешних и неуспјешних епизода, као и укупном трајању поступка.

7.10.6 Верификација

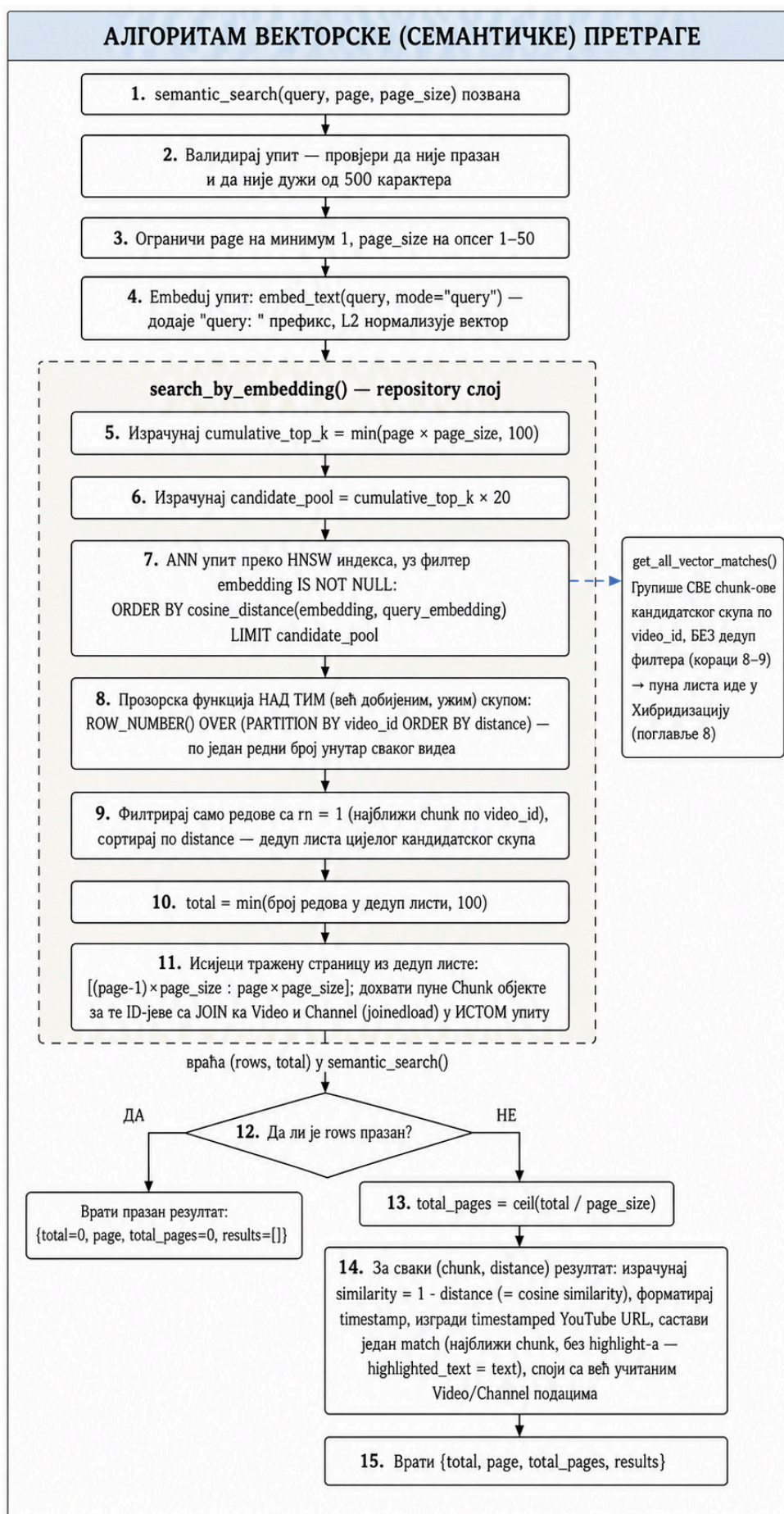
Након завршетка *batch embed*-овања, исправност поступка провјерена је на основу завршног стања *checkpoint* датотеке. Резултат обраде цијелог корпуса приказан је у листингу 19.

```
Успјешно обрађене епизоде: 15.889
Неуспјешне епизоде:      0
Генерисани chunk-ови:    337.991
```

Листинг 19: Резултат batch embedовања цјелокупног корпуса

7.11 Алгоритам претраге (online pipeline)

Док је претходни одјељак описао једнократни поступак попуњавања базе *embedding* векторима, **online pipeline** извршава се при свакој корисничкој претрази. Он обухвата обраду корисничког упита, генерисање његовог *embedding* вектора, *ANN* претрагу над *HNSW* индексом и формирање резултата који се враћају функцији за хибридизацију, који ће се појаснити у поглављу 8. Комплетан ток алгоритма претраге приказан је на слици 9.



Слика 9: Алгоритам векторске претраге

7.11.1 Embedовање упита

Функција `semantic_search()` прима текстуални упит и параметре за страничење (`page` и `page_size`). Прије извршавања претраге провјерава се исправност упита и параметара страничења, након чега започиње поступак семантичке претраге.

Након валидације, упит се претвара у *embedding* вектор позивом `embed_text(query, mode="query")`. Режим `"query"` додаје одговарајући `"query: "` префикс, након чега се вектор нормализује на исти начин као и вектори *chunk*-ова приликом *offline embed*-овања. Добијени вектор затим се користи за проналажење семантички најсличнијих дијелова транскрипта.

7.11.2 ANN претрага и груписање по епизоди

Претрага се извршава функцијом `search_by_embedding()` над *HNSW* индексом у *PostgreSQL* бази. Циљ је пронаћи *chunk*-ове чији су *embedding* вектори најближи вектору корисничког упита.

Пошто једна епизода садржи више *chunk*-ова, међу најближим резултатима може се појавити више *chunk*-ова истог видеа. Због тога је потребно осигурати да се свака епизода у коначној листи резултата прикаже само једном, са својим најрелевантнијим *chunk*-ом.

Директно груписање над цијелом табелом *chunks*, на примјер примјеном прозорске функције `ROW_NUMBER()` по `video_id`, захтијевало би израчунавање растојања над великим бројем редова прије груписања. Тиме би се изгубила главна предност *HNSW* индекса — брзо проналажење ограниченог скупа приближно најближих сусједа без потпуног прегледа табеле.

Због тога је примијењен приступ *overfetch* + дедупликација. *HNSW* индексом се прво дохвати шири кандидатски скуп најближих *chunk*-ова, а тек се над тим знатно мањим скупом резултати групишу према `video_id`. За сваки видео задржава се само његов најближи *chunk*, након чега се издваја страница резултата коју је корисник затражио.

Величина кандидатског скупа одређује се помоћу `CANDIDATE_MULTIPLIER = 20`. Ова вриједност је емпиријски одабрана на основу структуре коришћеног корпуса, у којем једна епизода у просјеку садржи приближно 21 *chunk*. Дохватањем већег броја кандидата смањује се ризик да велики број најближих *chunk*-ова припада истој епизоди и да након дедупликације не остане довољно различитих епизода за попуњавање тражене странице. Већи множилац би додатно повећао кандидатски скуп, али и трошак упита, па вриједност 20 представља практичан компромис између ова два захтјева.

Број резултата који се разматрају за страничење ограничен је вриједношћу `MAX_TOTAL = 100`. За разлику од лексичке претраге, код векторске претраге не постоји природна граница која одређује да ли нека епизода „одговара“ упиту — сваки вектор има одређено растојање од вектора упита, само су неки ближи од других. Због тога вриједност `total` не представља тачан број свих семантички повезаних епизода у бази, већ број дистинктних резултата пронађених унутар постављене границе `MAX_TOTAL`.

7.11.3 Формирање одговора

За сваки пронађени резултат израчунава се скор релевантности приказан у листингу 20.

```
similarity = round(1 - distance, 4)
```

Листинг 20: Израчунавање косинусне сличности на основу косинусног растојања

Пошто `cosine_distance` представља косинусно растојање, одузимањем те вриједности од 1 добија се косинусна сличност. Већа вриједност зато означава већу семантичку сличност између корисничког упита и текста *chunk*-а.

За сваку епизоду враћа се њен најрелевантнији *chunk*, заједно са текстом и временском ознаком. Вријеме почетка *chunk*-а (`start_sec`) претвара се у читљив формат помоћу функције `format_seconds()`, док се директан линк до одговарајућег тренутка у видеу формира додавањем `YouTube t=` параметра на `URL` епизоде.

За разлику од *Elasticsearch* претраге, семантичка претрага нема механизам за означавање подударних ријечи помоћу ознака. Разлог је сама природа векторске претраге: релевантност се одређује на основу семантичке сличности цјелокупног текста, а не директног поклапања појединачних ријечи. Због тога поље *highlighted_text* тренутно садржи исти текст као и поље *text*.

Метаподаци о епизоди, као што су наслов, канал и датум објаве, налазе се у истој *PostgreSQL* бази као и *chunk*-ови. Због тога се помоћу *joinedload()* дохватају заједно са резултатима претраге, без потребе за додатним упитом према другом систему. Ово се разликује од *Elasticsearch* претраге, код које је након претраге потребно засебно дохватити метаподатке из *PostgreSQL* базе.

7.11.4 Потпуна листа векторских погодака

Поред *semantic_search()*, имплементирана је и функција *get_all_vector_matches()*. Док *semantic_search()* задржава само најрелевантнији *chunk* сваке епизоде, ова функција враћа све пронађене *chunk*-ове унутар кандидатског скупа и групише их према *video_id*.

Она се не користи директно за приказ резултата кориснику, већ је намијењена хибридној претрази описаној у поглављу 8, гдје је потребно располагати са више релевантних временских тренутака унутар исте епизоде.

7.12 Ограничења векторске претраге

Векторска претрага омогућава проналажење семантички сличног садржаја чак и када упит и транскрипт не садрже исте ријечи. Ипак, управо због ослањања на значење умјесто на директно поклапање термина, она има ограничења у ситуацијама у којима је важан тачан облик текста. Због тога се векторска и лексичка претрага могу посматрати као комплементарни приступи. Предности једног приступа често надокнађују недостатке другог.

7.12.1 Тачно подударање

Embedding модел представља читав *chunk* једним вектором фиксне дужине. На тај начин задржава се семантичка информација о тексту, али се не чувају сви детаљи његовог тачног облика.

Због тога векторска претрага није најпогоднија када корисник тражи тачну фразу, име, број, датум или други садржај код којег је важно дословно подударање. Сличан, али не и идентичан текст може имати високу косинусну сличност са упитом. Особина која омогућава проналажење парафраза тако истовремено представља недостатак када је потребно пронаћи тачан облик текста.

Сличан проблем може се јавити код ријетких имена и скраћеница које су слабо заступљене у подацима на којима је *embedding* модел обучаван. Лексичка претрага у овим случајевима има предност јер директно пореди термине, а механизми као што је *fuzziness*: “*AUTO*” додатно омогућавају толеранцију на мање правописне грешке.

7.12.2 Означавање и објашњивост резултата

Elasticsearch може директно означити ријечи које су се подудариле са корисничким упитом. Код векторске претраге не постоји тако једноставна веза између појединачних ријечи упита и резултата, јер се пореде *embedding* вектори цјелокупног текста.

Због тога поље *highlighted_text* у тренутној имплементацији садржи исти текст као *text*, без означавања конкретног дијела *chunk*-а који је допринио резултату.

Ово уједно отежава објашњавање резултата. Док се *BM25* скор може повезати са конкретним терминима упита, косинусна сличност представља једну бројчану оцјену сличности два вектора. Систем зато може одредити да је један *chunk* семантички ближи упиту од другог, али не може једноставно показати које су ријечи или дијелови текста довели до таквог резултата.

7.12.3 Рачунарски трошак

Векторска претрага захтијева додатне ресурсе у односу на лексичку претрагу. *Embedding* модел мора бити учитан приликом генерисања вектора, а цијели корпус мора проћи кроз *offline embed*-овање прије него што семантичка претрага постане доступна. У овом раду тај поступак трајао је приближно 20 сати.

Промјена *embedding* модела захтијевала би поновно генерисање вектора за читав корпус. Ако нови модел користи другачију димензију вектора, потребно је прилагодити и одговарајућу колону у бази.

Додатно, параметри *HNSW* индекса (*m* и *ef_construction*) нису систематски оптимизовани за корпус коришћен у овом раду, већ су коришћене изабране вриједности без посебне експерименталне оптимизације. Њихово подешавање представља један од могућих праваца даљег рада.

7.12.4 Зависност од *embedding* модела и језика

Квалитет векторске претраге директно зависи од квалитета *embedding* модела и његове способности да представи српски језик. Пошто је *multilingual-e5-base* вишејезички модел, српски представља само један од језика обухваћених његовим тренингом.

То може бити посебно значајно код рјеђих језичких облика, регионалних варијанти и дијалеката који су потенцијално слабије заступљени у тренинг подацима. У оквиру овог рада није посебно испитивано колико такве варијације утичу на квалитет *embedding* репрезентација, па ово остаје отворено питање за будућу евалуацију.

7.13 Могућа унапређења

На основу наведених ограничења и искустава током имплементације, могу се издвојити следећи правци даљег развоја система:

- **Оптимизација *HNSW* индекса** - параметри *m* и *ef_construction* могли би се експериментално подесити за конкретан корпус и испитати њихов утицај на брзину и квалитет претраге.
- **Поређење *embedding* модела** - коришћени *multilingual-e5-base* могао би се упоредити са већим или новијим моделима, као што су *multilingual-e5-large* и *bge-m3*, уз мјерење односа између квалитета резултата и додатног рачунарског трошка.
- **Додатно рангирање резултата (енг. *re-ranking*)** - након *ANN* претраге, мањи скуп најбољих кандидата могао би се додатно рангирати прецизнијим моделом, чиме би се потенцијално побољшао квалитет коначних резултата.
- **Унапређење означавања релевантног текста** - тренутно се у векторској претрази не означава дио *chunk*-а који је најрелевантнији за упит. Додатни механизам могао би кориснику јасније показати због чега је одређени резултат приказан.
- **Семантичко *chunk*-овање** - умјесто ослањања искључиво на *Whisper* сегменте, границе *chunk*-ова могле би се одређивати и према промјенама значења или теме у тексту.
- **Проширивање упита** - ријетка имена, скраћенице и други специфични термини могли би се обрађивати проширивањем упита алтернативним облицима прије претраге.

Глава 8

Хибридизација резултата претраге (RRF)

У претходним поглављима описана су два приступа претрази истог корпуса: лексичка претрага заснована на *BM25* алгоритму (Глава 6) и семантичка претрага заснована на *embedding* векторима (Глава 7). Лексичка претрага даје предност поклапању термина из упита и документа, док семантичка претрага омогућава проналажење садржаја сличног значења и када не постоји потпуно поклапање коришћених ријечи. Због различитих начина одређивања релевантности, ова два приступа могу се међусобно допуњавати [41].

Луан и сарадници показују да *sparse* модели имају предност при прецизном препознавању поклапања термина, док *dense* модели боље препознају семантичку сличност између сродних појмова. Аутори такође показују да фиксне векторске репрезентације *dual-encoder* модела имају ограничења при прецизном претраживању дужих докумената и да комбиновање *sparse* и *dense* репрезентација може побољшати резултате претраге [41]. Комбиновање лексичке и семантичке претраге у јединствену ранг-листу представља хибридную претрагу [42].

У овом систему резултати лексичке и семантичке претраге обједињују се методом *Reciprocal Rank Fusion* (*RRF*). У наставку поглавља описани су принцип рада и разлози избора *RRF* методе, а затим и начин њене примјене у имплементираним систему.

8.1 Reciprocal Rank Fusion

Reciprocal Rank Fusion (*RRF*) је метода за спајање више ранжираних листи у јединствену ранг-листу [43]. За разлику од приступа који директно комбинују скорове различитих система, *RRF* се заснива искључиво на позицији резултата у свакој листи. Због тога није потребно претходно усклађивати, односно нормализовати, *BM25* скорове и вриједности косинусне сличности [43].

За документ (*d*) и скуп ранжираних листи (*R*), *RRF* скор дефинише се као [43]:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_{r(d)}}$$

гдје "*rank*"_{*r*}(*d*) представља позицију документа (*d*) у листи (*r*), док је (*k*) константа која ублажава утицај веома високих позиција. У имплементацији овог система, ако се епизода не појављује у једној од листи резултата, та листа једноставно не доприноси њеном *RRF* скору.

8.1.1 Улога константе (*k*)

Константа (*k*) смањује разлику у доприносу између резултата који се налазе на самом врху листе. Без ње би прва позиција доприносила (1), а друга (1/2), док са (*k* = 60) њихови доприноси износе (1/61) и (1/62). На тај начин једна веома висока позиција има мањи утицај на коначни резултат.

У оригиналном раду Согмаск и сарадници користили су (*k* = 60). Ова вриједност изабрана је током пилот-експеримента и задржана у каснијој евалуацији; аутори наводе да је била близу оптималне, али и да избор конкретне вриједности није био пресудан за резултате [43]. Иста вриједност коришћена је и у овом систему.

8.1.2 Интуиција кроз примјер

Претпоставимо да лексичка претрага за исти упит рангира епизоде (А), (В) и (С) на позиције 1, 2 и 3, док их семантичка претрага поставља на позиције 2, 4 и 1. За (*k* = 60) добија се:

$$RRF(A) = \frac{1}{60 + 1} + \frac{1}{60 + 2} = 0.03252$$

$$RRF(B) = \frac{1}{60 + 2} + \frac{1}{60 + 4} = 0.03175$$

$$RRF(C) = \frac{1}{60 + 3} + \frac{1}{60 + 1} = 0.03227$$

Конечан поредак је, према томе, ($A > C > B$). Епизоде (A) и (C) имају највећи скор јер су високо позициониране у обје листе. *RRF* на тај начин омогућава да резултат који је добро рангиран у више система добије допринос од сваког од њих, без директног поређења њихових оригиналних скорова.

8.2 Избор *RRF*-а у односу на алтернативне приступе

При избору начина спајања резултата разматрана су два алтернативна приступа: комбиновање оригиналних скорова лексичке и семантичке претраге и додатно рангирање резултата помоћу ***cross-encoder*** модела.

8.2.1 Комбиновање скорова

Један од начина спајања резултата јесте комбиновање ***BM25*** скорa и косинусне сличности, на примјер њиховом пондерисаном сумом. Међутим, ове вриједности нису директно упоредиве јер потичу из различитих модела рангирања и могу имати различите расподеле и опсеге. Због тога је при оваквом приступу потребно претходно нормализовати скорове, на примјер ***min-max*** нормализацијом, а затим одредити тежину сваког сигнала [44].

RRF избјегава потребу за усклађивањем скорова јер користи позицију резултата у појединачним ранг-листама, а не њихове оригиналне вриједности. Због једноставности и одсуства потребе за подешавањем тежина, овај приступ је погодан за систем развијен у оквиру овог рада.

8.2.2 *Cross-encoder reranking*

Друга разматрана могућност био је ***cross-encoder reranking***. Код овог приступа прво се постојећим механизмом претраге издваја мањи број кандидата, након чега се они додатно рангирају помоћу ***cross-encoder*** модела.

Разлика у односу на ***bi-encoder*** приступ коришћен у семантичкој претрази овог система јесте у начину обраде упита и текста. Код ***bi-encoder*** приступа упит и текст кодирају се независно у засебне ***embedding*** векторе, након чега се њихова сличност израчунава поређењем добијених вектора. ***Cross-encoder***, насупротив томе, прима упит и текст кандидата заједно и директно процјењује колико је тај текст релевантан за конкретан упит. Оваква директна интеракција омогућава прецизнију процјену релевантности, али захтијева засебну обраду сваког пара упита и кандидата, због чега је рачунски захтјевнија.

Истраживања показују да ***cross-encoder*** модели могу постићи добре резултате и у ***zero-shot*** условима, односно када се примјењују на податке из домена за који нису посебно обучавани или прилагођавани [45]. ***BEIR*** евалуација такође показује да модели засновани на поновном рангирању у просјеку постижу добре резултате у таквим условима, али уз веће рачунске трошкове [46].

Увођење ***cross-encoder*** модела у овај систем захтијевало би додатни корак обраде над кандидатима добијеним у првој фази претраге, као и избор и евалуацију модела погодног за српски језик и корпус коришћен у овом раду. Због тога је у тренутној имплементацији предност дата једноставнијем приступу заснованом на ***RRF***-у, док ***cross-encoder reranking*** представља могућност за будуће унапређење система.

8.2.3 Ограничења *RRF*-а

Иако је једноставан за примјену, ***RRF*** није нужно оптималан начин спајања резултата у сваком систему. Вгуч и сарадници показују да његови резултати могу зависити од избора параметара, као и да комбинација нормализованих лексичких и семантичких скорова може надмашити ***RRF*** када су доступни подаци за подешавање параметара фузије [47].

У оквиру овог рада **RRF** је ипак изабран јер не захтијева нормализацију различитих скорова нити податке за обучавање параметара фузије, а његова примјена над постојећим ранг-листама је једноставна и рачунски јефтина. Оптимизација начина спајања резултата на основу означеног скупа упита и релевантних епизода остаје могући правац даљег развоја система.

8.3 Архитектура — три ендпоинта

Систем излаже три одвојена ендпоинта за претрагу: **/search** за лексичку претрагу описану у поглављу 6, **/search/semantic** за семантичку претрагу описану у поглављу 7 и **/search/hybrid** за хибридную претрагу описану у овом поглављу. Оваква организација омогућава независно коришћење и тестирање сваког приступа, као и њихово директно поређење приликом евалуације система у поглављу 9.

Хибридна претрага имплементирана је у сервису **hybrid_search_service.py**, који користи постојеће функције **search_podcasts()** и **semantic_search()**. На тај начин избјегнуто је дуплирање логике лексичке и семантичке претраге, а њихови резултати се у хибридном сервису само обједињују и поново рангирају примјеном **RRF** методе.

Сва три ендпоинта користе исти формат одговора (**SearchResult** и **SearchResponse**), са пољима као што су *videoId*, *channelName*, *episodeTitle*, *relevanceScore* и *matches*. Захваљујући јединственом формату, **frontend** апликација и скрипта за евалуацију могу на исти начин обрађивати резултате сва три приступа претрази.

8.4 Ниво фузије резултата

Лексичка и семантичка претрага не извршавају се над истом јединицом текста. Лексичка претрага ради над појединачним сегментима транскрипта, док семантичка претрага ради над *chunk*-овима насталим груписањем више узастопних сегмената. Разлози за избор ових грануларности описани су у главама 6 и 7.

Због различите грануларности, резултате није практично директно спајати на нивоу сегмента или *chunk*-а. Један *chunk* може обухватати више сегмената, па резултати два система немају директно једнозначно пресликавање.

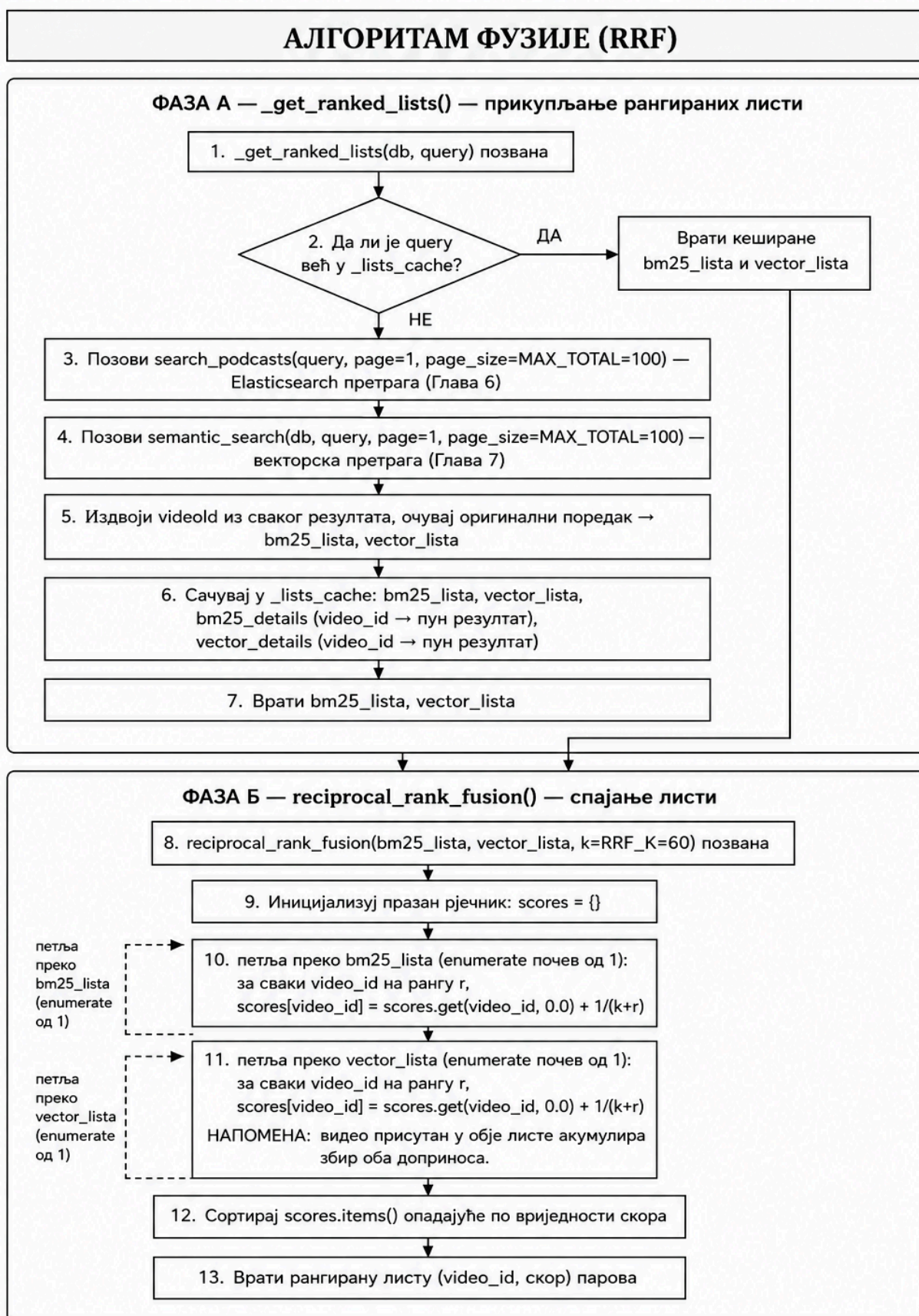
8.4.1 Фузија на нивоу епизоде

Резултати лексичке и семантичке претраге спајају се на нивоу епизоде, која представља њихов заједнички ниво. Сваки сегмент и сваки *chunk* припадају тачно једној епизоди и садрже њен *video_id*, што омогућава да се резултати прије примјене **RRF** методе групишу и рангирају према епизоди.

Овај приступ одговара и начину приказа резултата у корисничком интерфејсу, гдје основни резултат представља епизода, док пронађени сегменти и *chunk*-ови указују на релевантне дијелове њеног транскрипта.

8.5 Алгоритам фузије

Имплементација **RRF** методе подијељена је у два корака. Најприје се добијају рангиране листе епизода из лексичке и семантичке претраге, након чега се оне спајају у јединствену листу примјеном **RRF** формуле. Алгоритам је приказан на слици 10.



Напомена: Кеш приказан на дијаграму чува искључиво резултате претраге, независно од филтера. Примјена филтера над већ фузионисаном листом и засебан кеш за филтриране резултате описани су у одјелку 8.9.

Слика 10: Алгоритам фузије резултата примјеном *RRF* методе

8.5.1 Прикупљање ранжираних листи

Функција `get_ranked_lists()` позива `search_podcasts()` и `semantic_search()` са `page=1` и `page_size=MAX_TOTAL`, при чему је `MAX_TOTAL = 100`. На тај начин за фузију се прикупља до 100 најбоље ранжираних епизода из сваког система претраге, приказано у листингу 21.

```
def _get_ranked_lists(db: Session, query: str) -> tuple[list[str], list[str]]:
    cached = _lists_cache.get(query)
    if cached is not None:
        return cached["bm25_lista"], cached["vector_lista"]

    bm25_response = search_podcasts(query=query, page=1, page_size=MAX_TOTAL)
    vector_response = semantic_search(db, query=query, page=1, page_size=MAX_TOTAL)

    bm25_lista = [r["videoId"] for r in bm25_response["results"]]
    vector_lista = [r["videoId"] for r in vector_response["results"]]

    _lists_cache[query] = {
        "bm25_lista": bm25_lista,
        "vector_lista": vector_lista,
        "bm25_details": {r["videoId"]: r for r in bm25_response["results"]},
        "vector_details": {r["videoId"]: r for r in vector_response["results"]},
    }
    return bm25_lista, vector_lista
```

Листинг 21: Прикупљање и кеширање ранжираних листи *BM25* и векторске претраге

Из добијених резултата издвајају се `videoId` вриједности уз очување њиховог оригиналног поретка. Тако настају `bm25_lista` и `vector_lista`, које представљају улаз у *RRF* алгоритам. Остали подаци о резултатима чувају се у `bm25_details` и `vector_details` како би се касније могли искористити за формирање коначног одговора. Механизам кеширања ових података описан је у одјељку 8.8.

8.5.2 Спајање листи

Спајање двије ранг-листе реализовано је функцијом `reciprocal_rank_fusion()`, приказаном у листингу 22:

```
def reciprocal_rank_fusion(
    bm25_lista: list[str], vector_lista: list[str], k: int = RRF_K
) -> list[tuple[str, float]]:
    scores: dict[str, float] = {}

    for rank, video_id in enumerate(bm25_lista, start=1):
        scores[video_id] = scores.get(video_id, 0.0) + 1.0 / (k + rank)

    for rank, video_id in enumerate(vector_lista, start=1):
        scores[video_id] = scores.get(video_id, 0.0) + 1.0 / (k + rank)

    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Листинг 22: Имплементација Reciprocal Rank Fusion (RRF) алгоритма за спајање ранжираних листи

Функција пролази кроз обје листе и за сваку епизоду израчунава допринос ($1/(k+rank)$). Ако се иста епизода појављује у обје листе, њени доприноси се сабирају, док епизода присутна само у једној листи добија допринос искључиво на основу своје позиције у тој листи. Након обраде свих резултата, епизоде се сортирају опадајуће према добијеном *RRF* скору, чиме се формира јединствена ранг-листа хибридне претраге.

8.6 Пагинација хибридних резултата

Пагинација хибридне претраге примјењује се тек након спајања резултата. Просљеђивање параметара *page* и *page_size* директно лексичкој и семантичкој претрази није погодно, јер се коначни поредак епизода одређује тек након примјене *RRF* методе. Епизода која има нижу позицију у једној листи може након фузије бити високо рангирана захваљујући својој позицији у другој листи.

Због тога се за сваки нови упит најприје прикупља до ***MAX_TOTAL* = 100** најбоље ранжираних епизода из сваког система, приказаних у листингу 23:

```
bm25_response = search_podcasts(
    query=query,
    page=1,
    page_size=MAX_TOTAL
)

vector_response = semantic_search(
    db,
    query=query,
    page=1,
    page_size=MAX_TOTAL
)
```

Листинг 23: Покретање лексичке и векторске претраге за исти кориснички упит

Над тако добијеним листама извршава се *RRF* фузија, чиме настаје јединствена ранг-листа. Пагинација се затим примјењује над већ спојеним резултатима приказано у листингу 24.

```
total = len(rrf_rezultat)
offset = (page - 1) * page_size
stranica = rrf_rezultat[offset : offset + page_size]
```

Листинг 24: Пагинација резултата након *RRF* фузије

На овај начин све странице истог упита добијају се из истог кандидатског скупа, па се поредак резултата и вриједност *total* не мијењају приликом преласка са једне странице на другу. При томе *total* представља број јединствених епизода добијених фузијом прикупљених резултата, а не укупан број потенцијално релевантних епизода у читавом корпусу.

Овај приступ захтијева да се већ при првом позиву прикупи до 100 резултата из сваког система, чак и ако корисник прегледа само прву страницу. Тај компромис је прихваћен како би се обезбиједили стабилан поредак и досљедна пагинација, док се поновљени позиви за исти упит оптимизују механизмом кеширања описаним у наредном одјељку.

8.7 Кеширање

Како би се избјегло поновно извршавање лексичке и семантичке претраге приликом преласка између страница истог упита, прикупљене ранг-листе привремено се чувају у меморији. Кеш је имплементиран као *Python* рјечник.

Кључ представља текст корисничког упита, док се за њега чувају лексичка и семантичка ранг-листа (*bm25_lista* и *vector_lista*), као и подаци о резултатима потребни за касније формирање одговора. При поновном позиву са истим упитом ови подаци преузимају се из меморије, без поновног извршавања претраге у *Elasticsearch*-у и *PostgreSQL*-у.

У овој имплементацији коришћен је једноставан меморијски кеш умјесто посебног система као што је ***Redis***. Овакво рјешење довољно је за локално покретање и евалуацију система, а истовремено не уводи додатну инфраструктурну зависност.

Кеширају се изворне ранг-листе, а не коначна листа добијена *RRF* методом. На тај начин *RRF* скор се може поново израчунати над већ доступним резултатима, без новог извршавања лексичке и семантичке претраге.

Овако реализован кеш нема ограничење величине нити механизам истицања записа, а везан је за меморију појединачног *backend* процеса. Због тога представља рјешење прилагођено тренутном начину употребе система. За вишекорисничко или продукцијско окружење кеширање би се могло проширити ограничењем величине, политиком избацивања старих записа или употребом посебног система за кеширање.

8.8 Филтрирање након фузије

Поред текстуалног упита, систем подржава филтрирање резултата према називу епизоде, каналу и датуму објаве. У хибридној претрази филтери се примјењују након *RRF* фузије, односно над већ формираном ранг-листом епизода.

Лексичка и семантичка претрага зато се најприје извршавају без ових филтера, након чега се њихове ранг-листе спајају *RRF* методом. Филтери затим само уклањају епизоде које не задовољавају задате услове, без промјене међусобног поретка преосталих резултата.

Овај приступ омогућава да основни *RRF* ранг за исти текстуални упит остане непромијењен без обзира на накнадно изабране филтере. Да су филтери примјењивани прије претраге и фузије, свака комбинација филтера формирала би другачији кандидатски скуп, па би се мијењале и позиције епизода које улазе у *RRF* формулу.

Провера филтера реализована је функцијом `_passes_filters()`, приказано у листингу 25

```
def _passes_filters(details, episode_name, channel_name, date_from, date_to) -> bool:
    if episode_name and episode_name.strip().lower() not in details["episodeTitle"].lower():
        return False
    if channel_name and details["channelName"] != channel_name:
        return False
    if date_from or date_to:
        published_str = details.get("publishedAt")
        if not published_str:
            return False
        published_date = datetime.strptime(published_str, "%Y-%m-%d").date()
        if date_from and published_date < datetime.strptime(date_from, "%Y-%m-%d").date():
            return False
        if date_to and published_date > datetime.strptime(date_to, "%Y-%m-%d").date():
            return False
    return True
```

Листинг 25: Провера филтера над резултатима хибридне претраге

Филтер по називу епизоде заснива се на провери да ли се задати текст налази у називу, без употребе анализатора лексичке претраге. Филтер по каналу захтијева тачно поклапање назива канала, док се датум објаве проверава у односу на задату доњу и горњу границу. Ако датумски филтер постоји, а епизода нема познат датум објаве, она се искључује из резултата.

8.8.1 Кеширање филтрираних резултата

Исти текстуални упит може се користити са различитим комбинацијама филтера. Због тога се филтрирани резултати чувају одвојено од нефилтрираних ранг-листи, користећи кључ који обухвата упит и све параметре филтрирања.

Промјена било ког филтера формира нови кључ. Филтрирање се тада поново примјењује над већ кешираним резултатима претраге, без поновног позивања лексичког и семантичког система.

8.9 Претрага без текстуалног упита

Напредна претрага омогућава задавање само филтера, без текста упита. У том случају не постоји текстуални сигнал на основу којег би лексичка или семантичка претрага одређивале релевантност, па се *Elasticsearch* и векторска претрага не извршавају. Умјесто тога, систем директно претражује табелу *Video* у *PostgreSQL* бази и примјењује задате филтере према називу епизоде, каналу и датуму објаве. Ако нису задати ни текстуални упит ни филтери, враћа се празна листа резултата.

Пошто у овом случају не постоји скор релевантности, пронађене епизоде сортирају се према датуму објаве, од најновијих ка старијим. Формат одговора остаје исти као код осталих начина претраге, чиме се задржава јединствен начин обраде резултата на страни *frontend* апликације.

При филтрирању према горњој граници датума обухвата се читав дан наведен у параметру *date_to*. Пошто се у бази чува и вријеме објаве, као искључива горња граница користи се почетак наредног дана. На тај начин епизоде објављене током последњег дана задатог периода неће бити изостављене због времена објаве.

8.10 Приказ релевантних исјечака по епизоди

Хибридни резултат, поред ранга епизоде, садржи и временске исјечке (*matches*) који указују на дијелове транскрипта препознате као релевантне. Пошто лексичка и семантичка претрага ове исјечке проналазе независно, у хибридном одговору они се обједињују и приказују у јединственој листи.

Са лексичке стране користе се сегменти добијени кроз *Elasticsearch inner_hits*. Како би хибридни резултат могао приказати већи број релевантних дијелова исте епизоде, параметар *size* повећан је са 3 на 100. На тај начин може се вратити до 100 најбоље ранжираних сегмената за једну епизоду.

Са семантичке стране користи се функција *get_all_vector_matches()*, која, за разлику од поступка рангирања епизоде, не задржава само најбољи *chunk* по епизоди, већ групише све пронађене *chunk*-ове из кандидатског скупа према *video_id*.

8.10.1 Усклађивање векторског кандидатског скупа

Кандидатски скуп који користи *get_all_vector_matches()* усклађен је са скупом који се користи приликом семантичког рангирања. Његова величина зато се не задаје независно, већ се изводи из исте константе [26](#).

```
from app.repositories.chunk_repository import CANDIDATE_MULTIPLIER
```

```
VECTOR_CANDIDATE_POOL = MAX_TOTAL * CANDIDATE_MULTIPLIER
```

Листинг 26: Одређивање величине скупа кандидата за векторску претрагу

На овај начин промјена *CANDIDATE_MULTIPLIER* примјењује се на оба поступка, чиме се избегава неслагање између скупа *chunk*-ова коришћених за рангирање и скупа из којег се формирају исјечци за приказ.

8.10.2 Спајање и сортирање исјечака

За сваку епизоду лексички и семантички исјечци спајају се у једну листу и сортирају према временској позицији унутар видеа, приказано у листингу [27](#).

```
bm25_matches = bm25_details.get(video_id, {}).get("matches", [])
vector_matches = vector_matches_by_video.get(video_id, [])
```

```
combined = bm25_matches + vector_matches
combined.sort(key=lambda m: m["timestamp_seconds"])
```

```
result["matches"] = combined
```

Листинг 27: Спајање и хронолошко сортирање поклапања лексичке и векторске претраге

Корисник тако добија хронолошки уређен преглед релевантних дијелова епизоде, без обзира на то да ли су пронађени лексичком или семантичком претрагом.

Прикупљање додатних векторских исјечака захтијева још један упит над табелом *Chunk*. Добијени подаци кеширају се заједно са осталим резултатима упита, па се овај корак не понавља приликом преласка између страница исте претраге.

8.11 Ограничења и правци даљег развоја

Одређена ограничења семантичке претраге описана у поглављу 7 преносе се и на хибридни приступ. Прије свега, тренутна имплементација не користи минимални праг сличности за семантичке резултате, па и слабије повезани кандидати могу учествовати у *RRF* фузији. Одређивање одговарајућег прага на основу евалуационог скупа представља један од могућих праваца унапређења.

Вриједност параметра ($k = 60$) преузета је из оригиналног рада о *RRF* методи и није посебно оптимизирана за корпус коришћен у овом раду [43]. Даља евалуација могла би обухватити испитивање различитих вриједности овог параметра, као и других начина фузије. Посебно, комбинација нормализованих лексичких и семантичких скорова може надмашити *RRF* када су доступни подаци за подешавање параметара фузије [47]. Додатну могућност представља *cross-encoder reranking*, описан у одјељку 8.2.2.

Простор за унапређење постоји и у погледу скалабилности. Меморијски кеш нема ограничење величине нити механизам истицања записа, па би у вишекорисничком или продукцијском окружењу било потребно увести одговарајућу политику управљања кешом или посебан систем за кеширање. Такође, код већег корпуса требало би испитати однос између величине кандидатског скупа, времена одзива и квалитета хибридне претраге.

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	Преглед припреме података и фаза индексирања	7
Слика 2	Преглед обраде корисничког упита и формирања хибридних резултата	8
Слика 3	Дијаграм класа модела података: <i>Channel</i> , <i>Video</i> , <i>TranscriptSegment</i> и <i>Chunk</i>	16
Слика 4	Алгоритам увоза података из <i>.jsonl.gz</i> датотека у PostgreSQL базу.	17
Слика 5	Поједностављен приказ процеса анализе текста у Elasticsearch-у	23
Слика 6	Алгоритам индексирања у Elasticsearch-у — од провјере постојања индекса до верификације учитаних докумената	27
Слика 7	Алгоритам корисничке претраге у Elasticsearch-у	29
Слика 8	Алгоритам <i>batch embed</i> -овања корпуса и уписа векторских репрезентација у базу	39
Слика 9	Алгоритам векторске претраге	42
Слика 10	Алгоритам фузије резултата примјеном <i>RRF</i> методе	50

Списак листинга

Листинг 1	Конфигурација <i>PostgreSQL</i> базе са <i>pgvector</i> екстензијом	9
Листинг 2	<i>Dockerfile</i> за изградњу прилагођене <i>Elasticsearch</i> слике са <i>analysis-icu</i> додатком	10
Листинг 3	Конфигурација <i>nginx</i> -а за просљеђивање захтјева <i>Angular SPA</i> апликацији	10
Листинг 4	Команде за покретање и заустављање <i>Docker Compose</i> сервиса	11
Листинг 5	Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде	14
Листинг 6	Покретање скрипте за увоз података унутар <i>backend</i> контејнера	16
Листинг 7	Конфигурација мапирања <i>Elasticsearch</i> индекса	22
Листинг 8	Списак зауставних ријечи у уграђеној листи <i>serbian_stop</i>	24
Листинг 9	Конфигурација прилагођеног анализатора за српски језик	25
Листинг 10	Списак скраћеница коришћених у <i>Elasticsearch</i> претрази	26
Листинг 11	Резултат <i>bulk</i> индексирања докумената	28
Листинг 12	Конфигурација <i>match</i> упита за претрагу текста у <i>Elasticsearch</i> -у	30
Листинг 13	Дохватање метаподатака релационе базе	31
Листинг 14	Нормализација скорa релевантности и форматирање временских ознака сегмената	31
Листинг 15	Креирање <i>HNSW</i> индекса над <i>embedding</i> векторима	38
Листинг 16	Креирање <i>pgvector</i> екстензије у <i>Alembic</i> миграцији	38
Листинг 17	Покретање <i>batch embed</i> -овања корпуса унутар <i>backend</i> контејнера	40
Листинг 18	Спречавање дуплираног уписа <i>chunk</i> -ова у базу	40
Листинг 19	Резултат <i>batch embed</i> овања цјелокупног корпуса	41
Листинг 20	Израчунавање косинусне сличности на основу косинусног растојања	43
Листинг 21	Прикупљање и кеширање ранжираних листи <i>BM25</i> и векторске претраге	51
Листинг 22	Имплементација <i>Reciprocal Rank Fusion (RRF)</i> алгоритма за спајање ранжираних листи ...	51
Листинг 23	Покретање лексичке и векторске претраге за исти кориснички упит	52
Листинг 24	Пагинација резултата након <i>RRF</i> фузије	52
Листинг 25	Провјера филтера над резултатима хибридне претраге	53
Листинг 26	Одређивање величине скупа кандидата за векторску претрагу	54
Листинг 27	Спајање и хронолошко сортирање поклапања лексичке и векторске претраге	54

Списак табела

Табела 1	Списак канала у корпусу и број транскрибованих епизода по каналу.	13
----------	--	----

Списак коришћених скраћеница

Скраћеница	Опис
ANN	<i>Approximate Nearest Neighbor</i> (приближна претрага к-најближих суседа)
NLP	<i>Natural Language Processing</i> (обрада природног језика)
RRF	<i>Reciprocal Rank Fusion</i> (метода спајања ранжираних листи резултата)
TREC	<i>Text REtrieval Conference</i> (истраживачка конференција за проналажење текста)
ICU	<i>International Components for Unicode</i> (библиотека за нормализацију Unicode текста)
ASGI	<i>Asynchronous Server Gateway Interface</i> (стандард за асинхроне Python веб сервере)
ORM	<i>Object-Relational Mapping</i> (мапирање објектно-оријентисаних података у релациону базу)
SPA	<i>Single Page Application</i> (апликација чије се рутирање обавља на страни клијента)
ASR	<i>Automatic Speech Recognition</i> (аутоматско препознавање говора)
JSONL	<i>JSON Lines</i> (формат гдје сваки ред датотеке представља један самосталан JSON објекат)
TF-IDF	<i>Term Frequency-Inverse Document Frequency</i> (мјера важности термина у документу у односу на цијелу колекцију)
BM25	<i>Best Matching 25 — Okapi BM25</i> (алгоритам рангирања резултата претраге)
URL	<i>Uniform Resource Locator</i> (интернет адреса)
HNSW	<i>Hierarchical Navigable Small World</i> (графовски алгоритам/индекс за приближну претрагу к-најближих суседа)
nDCG	<i>Normalized Discounted Cumulative Gain</i> (мјера квалитета рангирања резултата претраге)

Списак коришћених појмова

Појам	Објашњење
Подкаст	Аудио (или аудио-видео) садржај који се дистрибуира у виду епизода, доступан за преузимање или стриминг путем интернета.
Семантичка претрага	Начин претраге који упоређује значење упита и садржаја, а не само подударање тачних термина.
Лексичка претрага	Начин претраге заснован на подударању тачних ријечи или термина између упита и садржаја (нпр. кроз индексирање кључних речи).
Хибридна претрага	Пристап претраживању који комбинује лексичку и семантичку претрагу ради постизања прецизнијих резултата.
Гранулација сегмената	Ниво детаљности на који је садржај подјељен (нпр. на један или више сегмената) ради навигације до тачног или приближног тренутка унутар епизоде.
Унакрсни језички пренос знања	Пренос научених представа или образаца из модела тренираног на једном (обично доминантном) језику на други, мање заступљен језик.
Референтни тест (енг. <i>benchmark</i>)	Стандардизован скуп задатака и података који омогућава упоредиво мјерење перформанси различитих система или модела.
Језик са ограниченим ресурсима	Језик за који постоји релативно мала количина дигитализованих текстуалних и говорних података, што отежава тренирање и прилагођавање модела обраде природног језика (насупрот доминантним језицима попут енглеског).
Корпус	Велика, структурисана збирка аутентичних језичких података (писаних или говорних), намјенски одабрана и узоркована тако да буде репрезентативна за одређени језик, често аотирана језичким метаподацима.
Морфологија	Део граматике који се бави врстама ријечи, њиховом структуром и промјеном њихових облика.
Лексема	Основна самостална јединица лексичког система која обухвата све граматичке облике и различита значења која једна ријеч може остварити.
Стематизација	Поступак свођења различитих облика ријечи на њихову заједничку основу (стем), уклањањем одређених наставака и дијелова ријечи; добијени стем не мора представљати стварну ријеч у језику.
Chunk	Спојени прозор од једног или два сегмента транскрипта, коришћен као јединица за генерисање векторског уграђивања (енг. <i>embedding</i>) у семантичкој претрази.
Векторско уграђивање (енг. <i>embedding</i>)	Нумеричка (векторска) репрезентација текста добијена моделом машинског учења, таква да семантички сличан садржај буде представљен блиским векторима у векторском простору.
Инвертовани индекс	Структура података у којој се, за разлику од класичног приступа, за сваки термин чува информација о документима у којима се тај термин појављује.
Индекс (Elasticsearch)	Логичка цјелина у <i>Elasticsearch</i> -у састављена од једног или више фрагмената, унутар које се физички чувају и индексирају документи.
Фрагмент (енг. <i>Shard</i>)	Физичка јединица у којој <i>Elasticsearch</i> складишти дио података једног индекса; свака представља засебну инстанцу претраживачког механизма.
Копија (енг. <i>Replica</i>)	Копија примарног фрагмента која служи као додатни ниво заштите података при отказу хардвера, а може учествовати и у обради захтјева за читање.

Чвор (енг. <i>Node</i>)	Покренута инстанца <i>Elasticsearch</i> -а.
Кластер (енг. <i>Cluster</i>)	Група чворова који заједно управљају подацима и ресурсима система.
Мапирање (енг. <i>mapping</i>)	Дефиниција структуре података у <i>Elasticsearch</i> -у која одређује тип и начин индексирања појединачних поља документа.
Анализатор (енг. <i>analyzer</i>)	Ланац трансформација кроз који пролази текст прије индексирања или претраживања, састављен од карактер филтера, токенизатора и филтера термина.
<i>Bulk API</i>	<i>Elasticsearch API endpoint</i> који омогућава извршавање више операција индексирања, ажурирања или брисања докумената у оквиру једног <i>HTTP</i> захтјева.
Идемпотентна операција	Операција која, ако се позове више пута са истим улазним подацима, даје исти резултат као да је позвана једном.
Вокабуларни јаз (енг. <i>vocabulary problem</i>)	Појава да двоје различитих људи, независно једно од другог, за исти појам или радњу бирају исти термин у мање од 20% случајева, услјед чега упит и садржај који описују исту тему често не дијеле ниједан заједнички термин.
Ријетка (sparse) векторска репрезентација	Векторска репрезентација текста код које већина координата има вриједност нула.
Густа (dense) векторска репрезентација	Векторска репрезентација текста код које су координате углавном различите од нуле, а њихова димензија није директно повезана са величином рјечника.
Унутрашњи производ (енг. <i>dot product</i>)	Мјера сличности два вектора која зависи и од угла између њих и од производа њихових дужина (норми), због чега вектори веће норме могу остварити већи унутрашњи производ без обзира на смјер.
Косинусна сличност (енг. <i>cosine similarity</i>)	Мјера сличности два вектора добијена нормализацијом унутрашњег производа дужинама оба вектора, тако да резултат зависи искључиво од угла између њих, а не од њихове дужине; узима вриједности у интервалу $[-1, 1]$.
Еуклидско растојање (енг. <i>Euclidean distance</i>)	Мјера удаљености два вектора у простору код које мања вриједност означава већу сличност; за разлику од косинусне сличности, зависи и од норми вектора, а не само од њиховог смјера.
Нормализација (вектора)	Поступак свођења дужине вектора на јединичну вриједност, при чему смјер вектора остаје непромијењен; над нормализованим векторима унутрашњи производ постаје еквивалентан косинусној сличности.
Тор-к претрага	Проблем проналажења k вектора из колекције који су најсличнији датом упитном вектору.
<i>Overfetch</i> и дедупликација	Стратегија претраге код које се преко индекса прво дохвата шири кандидатски скуп резултата, а тек се над тим знатно мањим скупом врши груписање и дедупликација по епизоди, чиме се избјегава губитак предности индекса услјед рада над цијелом табелом.
<i>Bi-encoder</i>	Приступ код којег се упит и текст кодирају независно у засебне <i>embedding</i> векторе, након чега се њихова сличност израчунава поређењем добијених вектора.
<i>Cross-encoder</i>	Модел који упит и текст кандидата обрађује заједно, директно процјењујући релевантност тог текста за дати упит; за разлику од <i>bi-encoder</i> приступа, овакав начин рада омогућава прецизнију процјену релевантности, али је рачунски захтјевнији и типично се примјењује за додатно рангирање (<i>reranking</i>) мањег броја претходно издвојених кандидата.
<i>Zero-shot</i> (услови/евалуација)	Примјена модела на податке из домена за који модел није посебно обучаван нити прилагођаван.

Фузија на нивоу епизоде	Приступ спајању резултата лексичке и семантичке претраге према заједничком идентификатору епизоде (<i>video_id</i>), примијењен умјесто спајања на нивоу појединачног сегмента или <i>chunk</i> -а, због различите грануларности на којој та два система претраге раде.
Пост-филтрирање	Приступ по којем се филтери примјењују тек над већ формираном, рангираном листом резултата, уклањајући оне који не задовољавају задате услове без промјене међусобног поретка преосталих резултата.

Биографија

Татјана Гавриловић је рођена 22.07.1999. године у Милићима, Република Српска, Босна и Херцеговина. Дјетињство је провела у Дервенти, гдје је 2014. године завршила основну школу “19. Април” као носилац Вукове дипломе. Исте године уписала је Гимназију општег смјера у Дервенти, коју је завршила 2018. године, поново као носилац Вукове дипломе и ученик генерације. Током школовања учествовала је на бројним општинским и регионалним такмичењима из математике, физике и историје.

У јулу 2018. године уписала је Факултет техничких наука у Новом Саду, смјер Софтверско инжењерство и информационе технологије, који је завршила 2022. године одбраном рада на тему “Имплементација Analyze me - Write me апликације употребом *Mendix* развојног окружења”.

Радно искуство стицала је кроз двије стручне праксе — у компанији Intermino у Новом Саду (јул – август 2022, Salesforce) и у компанији Vega IT (новембар 2022), на позицији софтверског инжењера у области веб програмирања. Од 2023. године запослена је у компанији Openfyle на позицији софтверског инжењера, гдје ради и данас. До 2025. године радила је и као сарадник у настави на Факултету техничких наука у Новом Саду, гдје је држала вјежбе из предмета Основе програмирања, Инжењерство серверског слоја и Пословне информатике — при чему је, слично као и на дипломском раду, у оквиру Пословне информатике коришћена *Mendix* развојна платформа.

Мастер студије, смјер Софтверско инжењерство и информационе технологије, уписала је у септембру 2023. године на Факултету техничких наука у Новом Саду. Говори енглески језик. Њена стручна интересовања обухватају системе за претрагу, модел-управљане (енг. *model-driven*) системе, системе засноване на знању и правилима (енг. *rule-based*), као и моделовање софтвера уопште. Интересовање за модел-управљене приступе дијелом произилази из искуства стеченог кроз рад са *Mendix* платформом, и на дипломском раду, у оквиру ког је развила апликацију намјењену писцима аматерима за подршку у писању књижевног романа, од почетне идеје до завршеног дјела.

Литература

- [1] A. Tamborrino, “Introducing Natural Language Search for Podcast Episodes.” Accessed: August 19, 2026. [Online]. Доступно на <https://engineering.atspotify.com/2022/03/introducing-natural-language-search-for-podcast-episodes>
- [2] J. Karlgren *et al.*, “TREC 2021 Podcasts Track Overview,” in *Proceedings of the Thirtieth Text REtrieval Conference (TREC 2021)*, National Institute of Standards, Technology (NIST), 2021. doi: [10.6028/NIST.SP.500-335.podcast-overview](https://doi.org/10.6028/NIST.SP.500-335.podcast-overview).
- [3] “TREC Podcasts Track.” Accessed: August 19, 2026. [Online]. Доступно на <https://trecpodcasts.github.io/>
- [4] “Семантичка претрага судске праксе.” Accessed: August 19, 2026. [Online]. Доступно на <https://sudskapraksa.rs/>
- [5] ICEF and R. Centar, “COMtext.SR.” Accessed: August 19, 2026. [Online]. Доступно на <https://icef-nlp.github.io/COMtext.SR/>
- [6] S. A. Ubois, “From Data Scarcity to Data Care: Reimagining Language Technologies for Serbian and other Low-Resource Languages,” *arXiv preprint arXiv:2512.10630*, 2025, doi: [10.48550/arXiv.2512.10630](https://doi.org/10.48550/arXiv.2512.10630).
- [7] V. Bataновић, “Методологија рјешавања семантичких проблема у обради кратких текстова написаних на природним језицима са ограниченим ресурсима,” Doctoral dissertation, 2020. Accessed: August 23, 2026. [Online]. Доступно на <https://nardus.mpn.gov.rs/bitstream/handle/123456789/17783/Disertacija.pdf?sequence=1&isAllowed=y>
- [8] M. Benjamin, “Hard Numbers: Language Exclusion in Computational Linguistics and Natural Language Processing.” Accessed: August 23, 2026. [Online]. Доступно на http://lrec-conf.org/workshops/lrec2018/W26/pdf/23_W26.pdf
- [9] L. Tang and T. Vuković, “NLP for preserving Torlak, a vulnerable low-resource Slavic language,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, 2025. Accessed: August 23, 2026. [Online]. Доступно на <https://aclanthology.org/2025.coling-main.423.pdf>
- [10] Elastic, “ICU analysis plugin.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu>
- [11] Одсек за стандардни језик, “Морфологија.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/morfologija/>
- [12] I. Klajn, *Gramatika srpskog jezika*. Beograd: Zavod za udžbenike i nastavna sredstva, 2005, p. 263. [Online]. Доступно на <https://archive.org/details/ivan-klajn-gramatika-srpskog-jezika/mode/2up>
- [13] Одсек за стандардни језик, “Лексема.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/leksema/>
- [14] Docker, “Defining and running multi-container applications with Docker Compose.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/compose/>
- [15] pgvector, “pgvector: Open-source vector similarity search for Postgres.” Accessed: August 23, 2026. [Online]. Доступно на <https://github.com/pgvector/pgvector>
- [16] Elastic, “Download X-Pack: Extend Elasticsearch and Kibana.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.elastic.co/downloads/x-pack>

-
- [17] FastAPI, “FastAPI documentation.” Accessed: August 23, 2026. [Online]. Доступно на <https://fastapi.tiangolo.com/>
- [18] Angular, “What is Angular?.” Accessed: August 23, 2026. [Online]. Доступно на <https://angular.dev/overview>
- [19] Docker, “Multi-stage builds.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/build/building/multi-stage/>
- [20] J. R. Batista, *Learn OpenAI Whisper: Transform your understanding of GenAI through robust and accurate speech processing solutions*. Packt Publishing, 2024, p. 372.
- [21] A. Jain, “TF-IDF in NLP (Term Frequency Inverse Document Frequency).” Accessed: August 21, 2026. [Online]. Доступно на <https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d>
- [22] S. Connelly, “Practical BM25 — Part 2: The BM25 Algorithm and its Variables.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables>
- [23] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide: A Distributed Real-Time Search and Analytics Engine*. O'Reilly Media, 2015. [Online]. Доступно на https://hlszny.com/booksAndPapers/buckets/b8_IT/elasticsearch-the-definitive-guide.pdf
- [24] Elastic, “How Full-Text Search Works.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works>
- [25] Elastic, “Similarity module.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html>
- [26] Elastic, “Language analyzers.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-lang-analyzer#serbian-analyzer>
- [27] Elastic, “ICU analyzer.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu-analyzer>
- [28] Elastic, “Stop token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stop-tokenfilter>
- [29] Apache Lucene, “Serbian stop words (stopwords.txt).” Accessed: August 21, 2026. [Online]. Доступно на <https://github.com/apache/lucene/blob/main/lucene/analysis/common/src/resources/org/apache/lucene/analysis/sr/stopwords.txt>
- [30] Elastic, “Stemmer token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stemmer-tokenfilter>
- [31] Elastic, “Bulk index or delete documents.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-bulk.html>
- [32] Elastic, “Tune for indexing speed.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/indexing-speed>
- [33] Elastic, “Match query.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/8.19/query-dsl-match-query.html>
- [34] N. Borwankar, *Vector Databases: A Practical Introduction*. O'Reilly Media, 2026. Accessed: August 26, 2026. [Online]. Доступно на https://books.google.com/books/about/Vector_Databases.html?id=25LOEQAAQBAJ

- [35] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais, “The Vocabulary Problem in Human-System Communication,” *Communications of the ACM*, vol. 30, no. 11, pp. 964–971, 1987, doi: [10.1145/32206.32212](https://doi.org/10.1145/32206.32212).
- [36] S. Bruch, “Foundations of Vector Retrieval,” *arXiv preprint arXiv:2401.09350*, 2024, doi: [10.48550/arXiv.2401.09350](https://doi.org/10.48550/arXiv.2401.09350).
- [37] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018, doi: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [38] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024, Accessed: August 26, 2026. [Online]. Доступно на <https://arxiv.org/abs/2402.05672>
- [39] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. Accessed: August 26, 2026. [Online]. Доступно на <https://aclanthology.org/2024.findings-acl.137/>
- [40] intfloat, “multilingual-e5-base.” Accessed: August 28, 2026. [Online]. Доступно на <https://huggingface.co/intfloat/multilingual-e5-base>
- [41] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, “Sparse, Dense, and Attentional Representations for Text Retrieval,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021, doi: [10.1162/tac1_a_00369](https://doi.org/10.1162/tac1_a_00369).
- [42] Elastic, “What is hybrid search? How it works and when to use it.” Accessed: August 29, 2026. [Online]. Доступно на <https://www.elastic.co/what-is/hybrid-search>
- [43] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Boston, MA, USA: ACM, 2009, pp. 758–759. doi: [10.1145/1571941.1572114](https://doi.org/10.1145/1571941.1572114).
- [44] OpenSearch, “Building effective hybrid search in OpenSearch: Techniques and best practices.” Accessed: August 29, 2026. [Online]. Доступно на <https://opensearch.org/blog/building-effective-hybrid-search-in-opensearch-techniques-and-best-practices/>
- [45] G. M. Rosa *et al.*, “In Defense of Cross-Encoders for Zero-Shot Retrieval,” *arXiv preprint arXiv:2212.06121*, 2022, Accessed: August 29, 2026. [Online]. Доступно на <https://arxiv.org/abs/2212.06121>
- [46] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021. Accessed: August 29, 2026. [Online]. Доступно на <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf>
- [47] S. Bruch, S. Gai, and A. Ingber, “An Analysis of Fusion Functions for Hybrid Retrieval,” *ACM Transactions on Information Systems*, 2023, doi: [10.1145/3596512](https://doi.org/10.1145/3596512).